General-BP: Improving the Framework for S-Box Circuits with Boyer-Peralta Algorithm-Based Heuristics

Anonymous Submission

Abstract. ~~The optimization of S-box circuits is crucial for efficient implementations of~~ In lightweight symmetric cryptography, ~~especially in resource-constrained environments where both circuit area (gate count) and latency (depth) must be minimized. In~~ circuits must be compact and fast. Gate count determines size, while depth determines speed. These two factors usually trade off against each other. Minimizing the number of gates while staying within a given depth is therefore practically important. Once nonlinear gate inputs and outputs are fixed, optimizing S-box ~~implementations, linear gates (XOR) typically constitute the majority of the total gate count, while nonlinear gates (AND) determine the cryptographic security properties. This structural characteristic makes optimizing linear components the principal target for reducing overall circuit complexity, as improvements in linear gate count and depth directly translate to significant gains in area and latency. Current optimization methods for linear gate implementations in~~ circuits reduces to linear-layer optimization. Both the optimization of S-boxes ~~suffer from limitations such as low computational efficiency , inadequate handling of depth constraints, and a lack of effective local optimization techniques.~~ with fixed nonlinear gates and linear layers can be solved by the Boyar-Peralta (BP) algorithm.

To unify the optimization of linear-layer circuits and S-boxes with fixed nonlinear gates, we present a detailed analysis of the BP algorithm and introduce a unified framework, General-BP. Building on this analysis, we identify several limitations of the BP algorithm, including low efficiency when solving depth-constrained problems, inflexible depth limits for nonlinear gate inputs, and issues arising from inaccurate or indistinguishable candidate evaluation scores. To address these ~~issues, this paper introduces an efficient algorithm based on the BP frameworkthat preserveslinear combinations of target vectors, enabling accelerated optimization of linear gates. We propose novel heuristic criteria for target vector selection and tie-breaking strategies under depth constraints, along with a new set representation for circuits that eliminates redundancies inherent in sequence-based and graph-based approaches. Our method includes advanced subset selection and local optimization techniques, further enhancing circuit quality. Applied to~~ limitations, we propose targeted improvements and integrate them into the General-BP framework. To enable fair comparison of BP algorithms and their variants, we propose a statistics-based metric. Using this metric, our enhanced strategies consistently outperform existing methods. When applying General-BP to the S-box circuits of `AES`~~, `SNOW3G`~~, `SNOW3G`, and `Saturnin`~~S-boxes, our approach achieves superior optimization results compared to existing work, with a 10, 000-fold improvement in computational efficiency~~, we achieve up to a 10^4-fold speedup over Jeon et al.'s method and obtain best-known results for all 36 benchmark cases. For the AES S-box under identical depth constraints, our circuits use 1-9 fewer gates than prior records.

Keywords: S-box · Low-latency implementation · Circuit depth · XOR Counts · the Boyar-Peralta Algorithm.

1 Introduction

~~Circuit minimization for components is a fundamental challenge in cryptographic implementation, directly impacting critical performance metrics like silicon area, power consumption, and operational speed in practical applications from IoT devices to server-grade hardware. The optimization process involves balancing two crucial parameters: gate count, which determines circuit area and static power, and gate depth, which defines computational latency. This inherent~~ Optimizing cryptographic component circuits is essential. Circuit optimization focuses on two aspects: *area* and *delay*. Area corresponds to *gate count*, and delay corresponds to *circuit depth*, with a typical trade-off ~~between circuit size and~~ relationship. Therefore, while optimizing gate count without depth constraints is meaningful, many practical applications, including automobiles, robotics, and other latency-sensitive systems, require circuits with low depth. Hence, it is of greater significance to study gate count optimization under depth ~~makes it essential to research sequencesthat achieve minimal gate counts under specific depth~~ constraints. ~~By constructing a portfolio of circuits for the same function with varying depth-size characteristics, designers are provided with a range of optimized options, facilitating the selection of the most appropriate circuit , depending on the specific application environment.~~

~~Linear~~ Extensive research on cryptographic circuit optimization has focused on linear layers and S-boxes~~are the two most critical components in symmetric cryptography. Consequently, much of the literature focuses on optimizing the circuit implementations of these components. For the circuit implementation of linear layers, the optimization of XOR gate count primarily relies on heuristic methods,~~ . Studies typically begin with the unconstrained gate-count minimization problem, then address the more challenging depth-constrained version. For linear-layer circuits, XOR count optimization mainly relies on heuristics such as the Paar algorithm [?] and the Boyar-Peralta (BP) algorithm [?]. ~~While the BP algorithm is slower than the Paar algorithm, it produces higher-quality solutions. Subsequent research has primarily presented improvements based on the BP algorithm, with most focusing on enhancements in its heuristic strategy [?, ?, ?]. Sun et al. [?] proposed a BP algorithm based on linear combinations, which significantly enhanced the efficiency of the BP algorithm. However, these BP algorithms did not consider depth constraints.Three main approaches have been introduced to optimize the depth-constrained circuit: the~~ Depth-constrained approaches include forward-search ~~method [?, ?, ?], the [?, ?, ?],~~ backward-search ~~method [?, ?], and a hybrid approach that combines both [?].~~

~~The implementation of S-boxes~~ [?, ?], and hybrid methods [?]. S-box optimization is more complex ~~to optimize due to the inclusion of~~ due to nonlinear gates. ~~For small-scale~~ Small-scale S-boxes ~~, current optimization primarily relies on~~ are often optimized using automation tools [?, ?, ?]~~. For~~ , while large-scale ~~S-boxes, such as 8×8 S-boxes, the optimization of their circuits are still challenging . The~~ ones (e.g., 8×8) remain challenging and commonly employ tower field decomposition ~~method [?, ?, ?] can be used to optimize the circuit implementation of large S-boxes. The theoretical foundation of this approach lies in the composite field theory of finite fields. In principle, any cryptographic component constructed based on operations in a finite field (particularly in GF(2^n)) can be decomposed and optimized using this method. For example, applying this method to the Advanced Encryption Standard (AES)~~ [?, ?, ?]. Since XOR gates typically outnumber nonlinear gates in S-box ~~enables a circuit implementation containing only 32 nonlinear gates [?].~~

~~In the circuit implementation of S-boxes, the number of XOR gates typically far exceeds that of AND gates. For example, [?] presented circuits for the S-Box of the AES that uses a total of 113 gates, of which 32 are nonlinear gates and 81 are linear gates. Therefore, optimizing the count of linear gates is crucial for reducing the overall circuit gate size. A framework presented in [?] focuses on optimizing the number of linear gates under a depth constraint for S-box circuits, while keeping AND gates fixed~~circuits, minimizing linear gates is crucial for reducing overall gate count.

The BP algorithm was first introduced by Boyar and Peralta. We refer to this original version as the *original BP* algorithm. Since then, numerous variants [?, ?, ?, ?, ?, ?] have been developed based on it. Currently, depth-constrained circuit optimization is mainly addressed using the BP algorithm. The original BP is a heuristic designed to optimize the number of `XOR` operations in linear-layer circuits. In circuit implementations, an `XOR` operation takes a pair of input nodes and produces an output node via `XOR`. Within the BP framework, nodes are typically represented as vectors. The original BP algorithm proceeds as follows:

1. A set is maintained to store the vectors that have already been generated. Initially, the set contains a collection of unit vectors. At each step, the algorithm expands the set by adding a newly generated vector.
2. At each step, for every candidate, the BP algorithm computes an objective function and assigns an *evaluation score* based on the value of objective function. This score is referred to as the *candidate evaluation score*. The algorithm then greedily selects the candidate with the highest score, generates the new vector, and adds it to the set.

~~However, the method proposed in [?] still has~~ 3. Each newly added vector reduces the objective function. After several steps, the objective function reaches zero, and the algorithm terminates.

Improvements to the BP algorithm have focused on refining candidate selection strategies, as proposed in [?, ?, ?]. More recently, new versions have prioritized efficiency [?]. These BP variants show promising results. However, our study finds considerable room for improvement. ~~First, while their~~ Tab. ?? summarizes areas for enhancement.

~~This paper presents advancements in depth-constrained circuit optimization algorithms, with three major contributions that improve both theoretical understanding and practical performance~~

Limitations and corresponding improvements. The explaination of the limitations in Tab. ?? are given as following:

~~First, we leverage the idea of preserving~~

Limitation 1. The original BP algorithm is inefficient because its objective function is computationally intensive. To address this, the LCB-BP algorithm [?], introduced in Sec. ??, speeds up the original BP by storing extra information, specifically linear combinations of ~~target vectorsfrom [?] and propose an efficient algorithm for linear gate optimization in circuit implementations. This~~ vectors, to avoid duplicated computations. However, while LCB-BP improves efficiency, it does not consider depth when computing these linear combinations, and therefore cannot guarantee that the generated circuits satisfy a given depth limit.

Contribution 1. We propose an algorithm ~~can be applied to S-box circuit optimizationwith improved efficiency. Based on this algorithm, we analyze the impact of depth constraints on minimal linear combinations of target vectorsand provide an interpretation of~~ for computing the objective function under a depth constraint. This includes methods for computing depth-constrained linear combinations and selecting only valid candidate pairs, thereby excluding those that would certainly exceed the depth limit. By storing those depth-constrained linear combinations, our method avoids duplicated computations. The proposed algorithm offers greater efficiency than approaches that do not store such information, such as [?]. Tab.?? shows that in optimizing the circuits of `AES`, `SNOW3G` and `Saturnin`, our method achieves a speedup of 10^4 compared to the ~~traditional BP algorithm 's maximum norm criterion from the perspective of linear combinations. Considering depth constraints, we introduce new heuristic criteria for target vector selection and novel tie-breaking strategies, which enhance solution quality. To better compare the effectiveness of different strategies~~state-of-the-art results in [?].

Limitation 2. In depth-constrained S-box optimization, input vectors of nonlinear gates

are assigned depth limits, which directly affect the number of candidate pairs available during construction. More candidates lead to a larger search space and a higher chance of better results. The existing depth assignment strategy is fixed: the initial limit remains throughout computation. If it is poorly chosen, the search space is unnecessarily restricted.

Contribution 2. We propose a dynamic depth-limit allocation. If a generated vector has a lower actual depth than its assigned depth limit, we can relax the depth limits of the remaining ungenerated vectors. This relaxation increases the number of available candidates in later steps, giving the algorithm greater flexibility to explore options and achieve better optimization. Applying this dynamic depth-limit allocation to the optimization of `AES`, `SNOW3G`, and `Saturnin` S-box circuits consistently yields improved results.

Limitation 3. BP's greedy mechanism always selects the pair with the highest evaluation score. The evaluation score assigned to each candidate estimates its actual effect, which cannot be computed since the circuit is not fully constructed. As a result, discrepancies between estimated and true contributions may lead the algorithm to make poor choices.
Contribution 3. We analyze the causes of inaccurate candidate evaluation scores and identify the specific situations where they are most likely to occur. To mitigate this problem, we propose a ~~new well-defined evaluation metric.~~ weighted scoring strategy that adjusts scores to better reflect each operation's true contribution. Experimental results demonstrate ~~the effectiveness of our proposed strategies under this metric. Based on the analysis of results, we provide recommendations for strategy selection under different depth bounds.~~ that this method contributes to finding better implementations of S-box circuits.

~~Secondly, we introduce a set representation method for circuits and, based on this representation, develop more effective approaches for selecting candidate subsets and performing local optimization. Our set representation avoids the redundancy issuespresent in sequence-based and tree/graph-based representations, thereby eliminating numerous redundant reconstruction operations. Besides, we present efficient local optimization methods under graph representation, making our local optimization approach more effective.~~
Limitation 4. When all candidates receive identical evaluation scores, it is impossible to distinguish between them, even though their actual effects may differ. Two heuristic approaches address this: (i) repeatedly running the algorithm with random tie-breaking to estimate the actual effect, and (ii) looking ahead several steps to include future consequences in the score. However, both methods are computationally expensive. As a result, when evaluation scores are indistinguishable, subcircuits generated are often suboptimal and lead to circuits with more operations than needed.
Contribution 4. We propose a more efficient subcircuit reconstruction method as a post-process step. The idea is to identify subcircuits that were generated under such indistinguishable scoring conditions and check whether a smaller set of operations can be found to replace the existing one. If such a replacement exists, the reconstructed circuit achieves a reduced operation count.

Contribution 5. We introduce a new evaluation metric to compare different strategies. The conventional approach compares different BP variants by their best results, i.e., the minimum gate count achieved. However, the occurrence of the best result is subject to randomness, making such comparisons unstable. Instead, we propose a statistics-based metric that yields stable and reliable comparisons. Using this metric, we demonstrate that both the dynamic depth-limit allocation strategy and the weighted scoring strategy consistently outperform existing strategies, particularly when the depth limit is low.

Finally, we apply ~~our approaches to optimize circuits for~~ the General-BP algorithm to optimize the S-box circuits of `AES`~~, `SNOW3G`~~, `SNOW3G`, and `Saturnin`~~S-boxes. For all circuits presented~~. Compared to all results reported in [**?**], we ~~achieve better optimization~~

~~resultsand compute circuit implementations at additional depth levels. Furthermore, compared to the algorithm in[?], our approach achieves a 10, 000-fold improvement in computational efficiency~~ obtain circuit implementations with fewer operations. Tab. ?? presents the best results obtained by General-BP for 36 instances, which also represent the current best-known results.

Table 1: Comparison of best results for `AES`, `SNOW3G`, and `Saturnin` S-box

S-box circuit	Depth	Gate count [?]	Gate count (this work)	S-box circuit	Depth	Gate count [?]	Gate count (this work)
`AES` S-box				`AES` S-box			
[?]	27	-	**112**	[?]	28	115	**113**
	26	113	**112**		19	-	**116**
	25	-	**113**		18	-	**119**
	24	-	**113**		17	-	**121**
	23	114	**113**		16	-	**127**
	22	117	**114**		15†	134	**127**
	21	122	**117**	Listings 3 [?]	19	-	**116**
	20	124	**119**		18	128	**121**
	19	-	**122**		17†	125	**123**
	18†	125	**123**	Listings 4 [?]	19	-	**118**
Listings 1 [?]	24	113	**112**		18	-	**120**
	23	115	**112**		17	129	**123**
	22	116	**112**		16†	134	**128**
	19	-	**116**	`SNOW3G` S-box			
	18	-	**120**	Listings 6 [?]	24	623	**579**
	17†	129	**121**		22†	-	**638**
Listings 2 [?]	19	-	**117**	`Saturnin` S-box			
	18	-	**121**				
	17	-	**123**				
	16†	137	**128**	[?]	25†	191	**171**

†: optimized depth for each circuit; -: not reported in [?].

Comparison with LCB-BP algorithm [?]. LCB-BP is the most recently proposed improvement to the BP algorithm. Here, we compare General-BP with LCB-BP. The key differences are as follows:

First, unlike General-BP, the LCB-BP algorithm [?] does not consider depth constraints. As a result, it may generate circuits whose depth exceeds the specified limit, making it unsuitable for depth-constrained circuit optimization. In contrast, our algorithm ensures that all generated circuits comply with depth constraints. Second, the LCB-BP algorithm focuses only on linear-layer optimization. However, S-box circuit optimization differs from linear-layer optimization, and LCB-BP cannot be directly applied to S-box circuits. Finally, although LCB-BP achieves efficiency improvements at the implementation level, it does not address the underlying limitations of the BP algorithm's greedy selection mechanism. In contrast, our work identifies and addresses these limitations, offering targeted improvements and insights for future research.

Structure of the ~~Paper~~paper. This paper is organized as follows. ~~Section ?? introduces necessary notations and preliminaries. Section ?? reviews existing BP algorithms and their main variants, along with local optimization methods. In Section ??, we present our new efficient algorithm for~~ Sec. ?? formulates the mathematical problem of depth-constrained circuit optimization and introduces the BP algorithm along with its variants. Sec. ?? presents a novel framework, termed the General-BP framework, for solving depth-constrained circuit optimization ~~and propose novel heuristic strategies. Besides, we introduces a new circuit representation method and provides approaches for subset selection and local optimization.~~ problems. Experimental results, analysis, and applications to the `AES`, `SNOW3G`, and `Saturnin` S-boxes are presented in ~~Section ??.~~Sec. ??. Finally, we conclude the paper in ~~Section~~ Sec. ??.

2 Preliminaries

In this section, we introduce the notations, formulates the mathematical problem of depth-constrained circuit optimization and introduces the BP algorithm along with its variants.

~~First, we present some notations used throughout the paper. Then, we introduce the core problems and key definitions of this paper.~~

2.1 Notations

Let $\mathbb{F}_2$ be the Galois Field GF(2) and $\mathbb{F}_2^n$ denote the n-dimensional vector space over $\mathbb{F}_2$. We use a lowercase boldface letter (e.g.~~$\mathbf{b} = (b_0, b_1, \cdots, b_n)$~~, $\mathbf{v} = (v_0, v_1, \ldots, v_{n-1})$) to represent a vector over $\mathbb{F}_2^n$. Let $\mathbf{e}_i$ ~~is~~ denote the unit vector with a one at the i-th position. ~~The~~ The zero vector is denoted by $\mathbf{0}$. The weight of a vector ~~$\mathbf{b}$ is a linear combination of the vectors $\mathbf{v}_0, \cdots, \mathbf{v}_l$ if $\mathbf{b} = c_0 \cdot \mathbf{v}_0 \oplus \cdots \oplus c_l \cdot \mathbf{v}_l$ where $c_i \in \mathbb{F}_2$. We~~ $\mathbf{v}$, denoted by $w(\mathbf{v})$, is the number of nonzero coordinates (i.e., the number of 1's) in $\mathbf{v}$. Let L be a set of vectors. If $\mathbf{v}_l = \bigoplus_{\mathbf{v} \in L} \mathbf{v}$, we say the set of vectors ~~$L = \{\mathbf{v}_i \mid c_i = 1\}$ is an linearrepresentation of $\mathbf{b}$, i.e., $\mathbf{b} = \bigoplus_{\mathbf{v} \in L} \mathbf{v}$ if~~ L is ~~an~~ a linear representation of ~~$\mathbf{b}$~~$\mathbf{v}_l$. The length of the linear representation L, denoted $|L|$, is the number of elements in L.

~~We use $\mathcal{B} = [\mathbf{b}_0, \mathbf{b}_1 \cdots, \mathbf{b}_l]$~~ We use $\vec{S}$ to denote a **sequence**, which is an ordered list of ~~vectors~~elements. The set of elements in ~~$\mathcal{B}$ is denote by $\mathrm{set}(\mathcal{B}) = \{\mathbf{b}_0, \mathbf{b}_1, \cdots, \mathbf{b}_l\}$. The notation $\mathbf{b}_i \in \mathcal{B}$ indicates that $\mathbf{b}_i$ is an element in the sequence $\mathcal{B}$. The notation $\mathcal{B} \preceq \mathcal{B}$ means that $\mathcal{B}'$ is a prefix of $\mathcal{B}$; that is, $\mathcal{B}'$ is a subsequence starting from the element at index 0. Formally, $\mathcal{B}' = [\mathbf{b}_0, \mathbf{b}_1, \cdots, \mathbf{b}_k]$ for some $k \leq l$. The length of a sequence $\mathcal{B}$~~ $\vec{S}$ is denoted by S. The length of $\vec{S}$ is the number of ~~elements in $\mathcal{B}$~~vectors it contains, denoted as ~~$|\mathcal{B}|$.~~ $|\vec{S}|$. For two sequences ~~$\mathcal{A} = [\mathbf{a}_0, \mathbf{a}_1, \cdots, \mathbf{a}_h]$ and $\mathcal{B} = [\mathbf{b}_0, \mathbf{b}_1 \cdots, \mathbf{b}_l]$~~$\vec{A} = [\mathbf{a}_0, \mathbf{a}_1, \ldots, \mathbf{a}_{h-1}]$ and $\vec{B} = [\mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{l-1}]$, the concatenation of ~~$\mathcal{A}$ and $\mathcal{B}$~~ $\vec{A}$ and $\vec{B}$ is denoted by ~~$\mathcal{A} \mid \mathcal{B} = [\mathbf{a}_0, \mathbf{a}_1, \cdots, \mathbf{a}_h, \mathbf{b}_0, \mathbf{b}_1 \cdots, \mathbf{b}$~~

2.2 ~~The SLP problem and the depth-constrainedSLP problem~~

$\vec{A} \mid \vec{B} = [\mathbf{a}_0, \mathbf{a}_1, \ldots, \mathbf{a}_{h-1}, \mathbf{b}_0, \mathbf{b}_1, \ldots, \mathbf{b}_{l-1}]$.

~~Boyar et al. studied the~~

2.2 Problem Formulation for Depth-Constrained Circuit Minimization

The circuit and the depth of circuit. A cryptographic circuit is a directed acyclic graph. Leaves are input nodes, and each internal node represents the result of an operation on its two children. Operations are either linear (e.g., `XOR`) or nonlinear (e.g., `AND`, `OR`). Other gates can be realized by combinations of these. The *depth* of a node is the longest path from any input to that node; inputs have depth 0. The *circuit depth* is the maximum depth of any output node.

Depth-constrained circuit minimization seeks, given a depth bound, a circuit with the fewest operations. Specifically, for a linear-layer circuit, this is the Shortest Linear Program (SLP) problem[?]~~, which is to minimize the number of linear operations necessary to compute a set of linear forms. Here, we use the mathematical description of the SLP problem given in [?]. Let K be a field and K^n be an n-dimensional vector space. Let $B^{(l)} = \{\mathbf{b}_0, \cdots, \mathbf{b}_{l-1}\}$ be~~. For S-box circuits with fixed nonlinear gates, minimizing the remaining linear part reduces to an SLP instance. These related minimization problems motivate a unified treatment. To provide a unified description of these two depth-constrained minimization problems, we introduce the following algebraic formalism.

The algebraic formalism of depth-constrained circuit minimization problem. Let U and V be two subspaces of $\mathbb{F}_2^n$ such that $U \cap V = \{\mathbf{0}\}$. Given a set of vectors ~~in K^n,~~

~~and T~~ X in U, and a linearly independent set of vectors $G = \{\mathbf{g}_0, \ldots, \mathbf{g}_{t-1}\}$ in V. Let $R = \{\mathbf{r}_0, \ldots, \mathbf{r}_{2t-1}\}$ be a set of vectors satisfying $\mathbf{r}_0, \mathbf{r}_1 \in \mathrm{Span}_{\mathrm{GF}(2)}(X)$ and $\mathbf{r}_{2i}, \mathbf{r}_{2i+1} \in \mathrm{Span}_{\mathrm{GF}(2)}(X \cup \{\mathbf{g}_0, \ldots, \mathbf{g}_{i-}$ for $i = 1, \ldots, t-1$. Let Y be a ~~set of target vectors in $\mathrm{Span}_K(B^{(l)})$, where $\mathrm{Span}_K(B^{(l)}) = \{\sum c_i \cdot \mathbf{b}_i \mid c_i \in K, \mathbf{b}_i \in$~~ set of vectors in $\mathrm{Span}_{\mathrm{GF}(2)}(X \cup G)$. The vectors in X and Y correspond to the initial inputs and outputs of the circuit, respectively. The sets R and G are the input and output nodes of the nonlinear gates: each nonlinear gate takes $\mathbf{r}_{2i}$ and $\mathbf{r}_{2i+1}$ as inputs and produces $\mathbf{g}_i$ as output. In practice, we usually take vectors in X and G to be unit vectors.

~~A sequence of vectors $\mathcal{B}^{(s)} = [\mathbf{b}_l, \mathbf{b}_{l+1}, \cdots, \mathbf{b}_{l+s-1}]$is called a **vector sum sequence** with respect to $B^{(l)}$ if it satisfies:~~

~~1. for each k with $l \leq k < l+s$, there exist indices $i, j < k$ such that $\mathbf{b}_k = \mathbf{b}_i + \mathbf{b}_j$, where $+$ denotes the vector addition in K^n.~~

~~A vector sum sequence is called a~~ ~~for T with respect to $B^{(l)}$ if it additionally satisfies:~~

~~2. $T \subseteq \{\mathbf{b}_l, \cdots, \mathbf{b}_{l+s-1}\}$.~~

~~The problem is how to find a for T with respect to $B^{(l)}$ that has the minimal length.~~

~~In this paper, we refer to the as the CVS sequencefor brevity. Minimizing the number of XOR gates in circuit design is equivalent to solving the SLP problem over the binary field $\mathbb{F}_2$, which corresponds to computing a CVS sequence with minimal length. Specifically, for a binary matrix $M_{m \times n}$ over $\mathbb{F}_2$, each row of the matrix can be viewed as a target vector. Let $T = \{\mathbf{t}_0, \mathbf{t}_1, \cdots, \mathbf{t}_{m-1}\}$ be the **target set** consisting of these target vectors. A natural and common choice for the initial **input generator set** $B^{(l)}$ is the set of unit vectors in $\mathbb{F}_2^n$, which has size $l = n$. In this case, we have $l = n$. And the vectors in $B^{(l)}$ are called input vectors. The SLP problem then reduces to finding the minimal~~ We aim to construct a sequence of vectors ~~generated by pairwise XOR operations that includes all target vectors in T. In the context of circuit implementation, each vector in the sequence corresponds to a node in the circuit, which is computed as the XOR of two previous nodes (vectors). The number of vectors in the corresponds directly to the number of XOR gates required in the circuit implementation.~~

~~It is important to note that a for a set of target vectors does not have a one-to-one correspondence with its circuit implementation. When generating an element $\mathbf{b}_k$ in a , a specific algorithm selects predecessors $\mathbf{b}_i$ and $\mathbf{b}_j$ such that $\mathbf{b}_k = \mathbf{b}_i + \mathbf{b}_j$~~ $\vec{S}$ that contains all vectors in Y. The sequence $\vec{S}$ is built iteratively: each new vector $\mathbf{w}$ is obtained by applying an operation $\mathtt{OP}$ to $\mathbf{u}$ and ~~$i, j < k$. However, at the point of generating $\mathbf{b}_k$, there may exist other available predecessor pairs $(\mathbf{b}_u, \mathbf{b}_v)$ that also satisfy $\mathbf{b}_k = \mathbf{b}_u + \mathbf{b}_v$ with $u, v < k$.~~

~~For example, given input vectors $\mathbf{b}_0 = (1,0,0)$, $\mathbf{b}_1 = (0,1,0)$, $\mathbf{b}_2 = (0,0,1)$, and a $[\mathbf{b}_3, \mathbf{b}_4, \mathbf{b}_5]$~~ $\mathbf{v}$, i.e., $\mathbf{w} := \mathtt{OP}(\mathbf{u}, \mathbf{v})$ where ~~$\mathbf{b}_3 = (1,1,0)$ (generated as $\mathbf{b}_0 \oplus \mathbf{b}_1$), $\mathbf{b}_4 = (0,1,1)$ (generated as $\mathbf{b}_1 \oplus \mathbf{b}_2$), and the target vector is $\mathbf{b}_5 = (1,1,1)$. Then, $\mathbf{b}_5$ can be generated in two ways:~~

~~$\mathbf{b}_5 = \mathbf{b}_3 \oplus \mathbf{b}_2$ (since $(1,1,0) \oplus (0,0,1) = (1,1,1)$)~~

~~$\mathbf{b}_5 = \mathbf{b}_0 \oplus \mathbf{b}_4$ (since $(1,0,0) \oplus (0,1,1) = (1,1,1)$)~~

~~Consequently, a single can correspond to multiple distinct circuit implementations.~~

~~(Depth) Let $B^{(l)}$ be the and T be the . $\mathcal{B}^{(s)}$ is a~~ $\mathbf{u}$ and $\mathbf{v}$ are vectors in X or already appear in $\vec{S}$. Here, the operation $\mathtt{OP}$ can be one of the following two types: linear operation and nonlinear operation. The linear operation is the $\mathtt{XOR}$ operation , i.e., $\mathbf{w} := \mathbf{u} \oplus \mathbf{v}$. We use $\mathtt{NL}_i$, for ~~T with respect to $B^{(l)}$. The depth~~ $i = 0, \ldots, t-1$, to represent the fixed nonlinear operation, i.e., the input must be $\mathbf{u} = \mathbf{r}_{2i}$ and $\mathbf{v} = \mathbf{r}_{2i+1}$, and the output of $\mathtt{NL}_i(\mathbf{u}, \mathbf{v})$ is $\mathbf{w} := \mathbf{g}_i$.

The *depth* of a vector ~~$\mathbf{b} \in \mathcal{B}^{(s)}$ is defined as the length of the longest path from an input vector to $\mathbf{b}$, denoted by $\mathcal{D}(\mathbf{b})$~~is denoted by $\mathcal{D}(\cdot)$. The depth of ~~the sequence $\mathcal{B}^{(s)}$ is~~

~~defined as the maximum depth of all target vectors denoted by $\mathcal{D}(\mathcal{B}^{(s)})$, that is~~

$$\mathcal{D}(\mathcal{B}^{(s)}) = \max\{\mathcal{D}(\mathbf{t}) \mid \mathbf{t} \in T\}.$$

every vector in X is 0. For a vector ~~$\mathbf{b}_k$ in a $\mathcal{B}$, suppose there exist multiple pairs of predecessor vectors $\{\mathbf{b}_{i_0}, \mathbf{b}_{j_0}\}, \cdots, \{\mathbf{b}_{i_u}, \mathbf{b}_{j_u}\}$ such that $\mathbf{b}_k = \mathbf{b}_{i_v} \oplus \mathbf{b}_{j_v}$ and $i_v, j_v < k$ for $0 \le v \le u$. Then the depth of vector $\mathbf{b}_k$ is defined as:~~

$$\mathcal{D}(\mathcal{B}, \mathbf{b}_k) = \min\{\max\{\mathcal{D}(\mathbf{b}_{i_v}), \mathcal{D}(\mathbf{b}_{j_v})\} | 0 \le v \le u\} + 1.$$

~~The depths of the input vectors $\mathbf{b}_0, \ldots, \mathbf{b}_{l-1}$ are typically defined as 0~~$\mathbf{w}$ in $\vec{S}$ generated by $\mathbf{w} := \mathtt{OP}(\mathbf{u}, \mathbf{v})$, the depth $\mathcal{D}(\mathbf{w})$ is $\max(\mathcal{D}(\mathbf{u}), \mathcal{D}(\mathbf{v})) + 1$. The depth of the whole sequence $\vec{S}$ is $\mathcal{D}(\vec{S}) = \max\{\mathcal{D}(\mathbf{w}) \mid \mathbf{w} \in \vec{S}\}$. ~~When there is no ambiguity, we denote $\mathcal{D}(\mathcal{B}, \mathbf{b}_k)$ as $\mathcal{D}(\mathbf{b}_k)$.~~

~~The Depth-Constrained Shortest Linear Program (DC-SLP) problem~~ *The problem of depth-constrained circuit minimization with fixed nonlinear gates* is to find a ~~CVS sequence that minimizes the sequence length while satisfying the depth constraint.~~ sequence $\vec{S}$ of minimum length that satisfies the above conditions and $\mathcal{D}(\vec{S}) \le H$, where H is a given depth limit When $R, G = \emptyset$, the problem reduces to minimizing a circuit of linear layers.

2.3 ~~The framework of Jeon et. al. for generating S-Box Circuits~~

~~The framework of Jeon et. al. [?] was presented at CHES 2025. They introduces a novel heuristic approach for generating optimized S-box circuits, which has three stages: pre-processing, optimization, and post-processing. In the pre-processing stage, the original S-box circuit is analyzed to extract XOR-based relationships and is transformed into multiple equivalent circuit representations by applying properties of nonlinear gates, thereby increasing the search space for potential optimizations. The core optimization stage incorporates the extended BP-based heuristics, such as the eBPD (extended Boyar-Peralta with Depth limit) algorithm. This stage optimizes the entire S-box circuitas a whole, rather than separating the small S-boxes and linear components. Finally, a post-processing step manages NOT gates that were removed earlier, potentially merging them into XNOR operations.~~ Each vector in the sequence $\vec{S} = [\mathbf{w}_0, \ldots, \mathbf{w}_{l-1}]$ corresponds to a node in the circuit, thus $\vec{S}$ is called the *node sequence* of (Y, G, R) with respect to X. Every node sequence has an associated *operation sequence* $\vec{S}_{\mathtt{OP}} = [(\mathbf{w}_0, \mathbf{u}_0, \mathbf{v}_0), \ldots, (\mathbf{w}_{l-1}, \mathbf{u}_{l-1}, \mathbf{v}_{l-1})]$, where the triple $(\mathbf{w}_i, \mathbf{u}_i, \mathbf{v}_i)$ indicates that $\mathbf{w}_i$ is generated as $\mathbf{w}_i := \mathtt{OP}(\mathbf{u}_i, \mathbf{v}_i)$ for $i = 0, \ldots, l-1$.

~~They focuses exclusively on optimizing the count and depth of linear gates and does not address the optimization of nonlinear gate implementations. For S-boxes, once the specific algebraic expression determining the nonlinear gates is fixed, the subsequent circuit optimization task reduces primarily to optimizing the linear operations.~~

We use the ~~extended to represent the an implementation of an~~ following example to illustrate the concepts and methods in this paper.

3 ~~State-of-art heuristics~~

2.1 ~~The BP Algorithm and Its Variants~~

2.1 BP algorithm and its variants

~~The~~ **Original BP** ~~**Algorithm**~~ **algorithm.** The Boyar-Peralta ~~algorithm (BPalgorithm) [?] is a heuristic greedy algorithm and it builds a $\mathcal{B}$ of base vectors step by step. At~~ (BP)

algorithm [?] was originally developed for optimizing the XOR count in linear-layer circuits. It maintains a set S of vectors. Initially, S contains the input vectors from X. At each step, the algorithm selects a pair of vectors from a candidate set. It generates a new vector by XORing them.

To construct the candidate set at each step, ~~$\mathcal{B}$ is extended by concatenating a new vector, potentially reducing the distance to the target vectors. Given a set of vectors B and~~ consider all unordered pairs of distinct vectors from the current set S, defined as $\{\{\mathbf{w}_i, \mathbf{w}_j\} \mid \mathbf{w}_i, \mathbf{w}_j \in S, \mathbf{w}_i \neq \mathbf{w}_j\}$. For each candidate pair $\{\mathbf{w}_i, \mathbf{w}_j\}$, generate a possible new set $S_{ij} = S \cup \{\mathbf{w}_i \oplus \mathbf{w}_j\}$. Then, for every target vector $\mathbf{y} \in Y$, compute its distance to the set S_{ij}.

The distance of a target vector ~~$\mathbf{t}$, the distance from B to $\mathbf{t}$, denoted by $\delta(B, \mathbf{t})$, is~~ $\mathbf{y}$ from a set S is defined as the minimum number of ~~operations required to compute $\mathbf{t}$ starting~~ XOR operations needed to generate $\mathbf{y}$ from vectors in ~~B. Here, we give the definitions of distance~~ S. More formally,

$$\delta(S, \mathbf{y}) = \min\{|L| - 1 \mid L \subseteq S \text{ and } \oplus_{\mathbf{v} \in L} \mathbf{v} = \mathbf{y}\}. \tag{1}$$

For each candidate set S_{ij}, we compute its distance vector: $(\delta(S_{ij}, \mathbf{y}_0), \ldots, \delta(S_{ij}, \mathbf{y}_{m-1}))$, and total distance: $totalDist = \sum_{k=0}^{m-1} \delta(S_{ij}, \mathbf{y}_k)$. Here, the total distance serves as the objective function. At each step, the candidate pair that yields the smallest total distance is selected. If there exists a target vector $\mathbf{y}$ such that its distance from S_{ij} is 1, then the pair $\{\mathbf{w}_i, \mathbf{w}_j\}$ is selected immediately, regardless of the total distance sum. This situation is referred to as a "pre-emptive" case.

~~(Distance)Given a target vector$\mathbf{t}$ and a set of vectors B, the set of all linear representations of $\mathbf{t}$ with respect to B is defined as :~~

$$\cancel{\mathrm{LinSet}(B, \mathbf{t}) = \{L \mid \mathbf{t} = \bigoplus_{\mathbf{b} \in L} \mathbf{b},\ L \subseteq B\}.}$$

~~The *distance* from B to $\mathbf{t}$ is defined as the minimal length of all linear representations minus one:~~

$$\cancel{\delta(B, \mathbf{t}) = \min\{|L| - 1 \mid L \in \mathrm{LinSet}(B, \mathbf{t})\}.}$$

~~The BP algorithm establishes the criterion for selecting vector pairs based on distance , and its main procedure is outlined as follows.~~

~~1. Enumerate all pairs of basis vectors $\{\mathbf{b}_i, \mathbf{b}_j\}$ and obtain a new vector $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$~~ At each step, a new vector is generated: $\mathbf{w}_{new} := \mathbf{w}_i \oplus \mathbf{w}_j$ where ~~$\mathbf{b}_i, \mathbf{b}_j \in B, i < j$. For each pair , compute the distance sum $TotalDist = \sum_{\mathbf{t} \in T} \delta(B \cup \{\mathbf{b}_k\}, \mathbf{t})$.~~

~~2. Select the vector $\mathbf{b}_k$ that minimizes the distance sum $TotalDist$, and add it to the current sequence $\mathcal{B}$~~$\{\mathbf{w}_i, \mathbf{w}_j\}$ is the selected pair. The vector $\mathbf{w}_{new}$ is then added to S. This process repeats until $Y \subset S$.

~~3. In case of tie, choose the vector that maximises the Euclidean norm of distance, i.e., select $\mathbf{b}_k$ corresponding to $\max |Dist(B \cup \{\mathbf{b}_k\})|_2$ where $Dist(B \cup \{\mathbf{b}_k\}) = (\delta(B \cup \{\mathbf{b}_k\}, \mathbf{t}_0), \ldots, \delta(B \cup \{\mathbf{b}_k\}, \mathbf{t}_{m-1}$ and $|Dist(B \cup \{\mathbf{b}_k\})|_2 = \sqrt{\sum_{i=0}^{m-1} \delta(B \cup \{\mathbf{b}_k\}, \mathbf{t}_i)^2}$ for $\mathbf{t}_i \in T$. (Norm criterion)~~

Linear-Combinations-based BP (LCB-BP) algorithm.

~~**The LCB-BP Algorithm** [?] presented the linear combinations based BP algorithm to improve the efficiency of~~ The Linear-Combinations-based BP (LCB-BP) algorithm [?] is more efficient than the original BP and the best results of LCB-BP are comparable to those of the BP algorithm. ~~They found that the most time-consuming aspect of the Boyar-Peralta (BP) algorithm is the computation of distances~~The main difference

between LCB-BP and BP lies in how the objective function is computed. Specifically, ~~for every possible pair of vectors generated so far, the algorithm must consider the new vector formed by their XOR and then recompute the distance for every target vector relative to the updated set~~ LCB-BP computes and stores the minimal linear representations of each target vector with respect to the current set of vectors S. A minimal linear representation of $\mathbf{y}$ with respect to S is a linear representation whose length equals $\delta(S, \mathbf{y}) + 1$; in other words, it is a representation that achieves the minimal length. ~~This exhaustive pairwise evaluation and repeated distance calculation create a significant computational bottleneck.~~

~~To enhance the efficiency of the algorithm,they leveraged the relationship between the minimal linear combinations of target vectors and their distances. As illustrated in ??,~~ A pair $\{\mathbf{w}_i, \mathbf{w}_j\}$ appears in a linear representation L if both $\mathbf{w}_i, \mathbf{w}_j \in L$. For any such pair in a minimal linear representation of a target vector $\mathbf{y}$, LCB-BP proves the following property. If we add $\mathbf{w}_i \oplus \mathbf{w}_j$ to S to get S_{ij}, then the distance from S_{ij} to $\mathbf{y}$ is always one less than the distance from S to $\mathbf{y}$. This is because a minimal linear representation of $\mathbf{y}$ with respect to S_{ij} can be directly generated from L: $L' = (L \setminus \{\mathbf{w}_i, \mathbf{w}_j\}) \cup \{\mathbf{w}_i \oplus \mathbf{w}_j\}$. Based on this property, candidates in LCB-BP are limited to those appearing in the ~~distance of a target vector is defined as the minimum number of XOR operations required to express it as a linear combination of the vectors generated so far. Specifically, by precomputing the~~ minimal linear ~~combinations for each target vector, they determined which pairs of existing vectors could reduce the distance of each target. These candidate pairs were then recorded in support sets. By counting the frequency of each pair across the support sets of all target vectors, they could identify the pair that maximally reduces the $TotalDist$.~~ representations. More precisely, the candidate set is $\cup_{\mathbf{y} \in Y} \text{support}(\mathbf{y})$, where $\text{support}(\mathbf{y})$ lists all pairs in the minimal linear representations of $\mathbf{y}$.

For each pair $\{\mathbf{w}_i, \mathbf{w}_j\}$, LCB-BP counts how many target vectors have this pair in their minimal linear representations. This *frequency* is defined as: $\text{frequency}(\{\mathbf{w}_i, \mathbf{w}_j\}) = |\{\mathbf{y} \in Y \mid \exists L \in \mathcal{L}(\mathbf{y})$ such that where $\mathcal{L}(\mathbf{y})$ denotes the set of all minimal linear representations of $\mathbf{y}$. This count corresponds to the reduction in the total distance sum achieved in the BP algorithm. At each step, we select the pair with the largest count. We then add the resulting vector $\mathbf{w}_{new} := \mathbf{w}_i \oplus \mathbf{w}_j$ to the set S. This approach significantly ~~improves the performance of the BP algorithm by avoiding the computationally expensive enumeration of all possible pairs and the calculation of distances~~ reduces the number of candidate pairs that need to be evaluated.

To interpret these frequencies, note that for each pair, its frequency serves as an estimate of the pair's actual contribution, and we refer to this as the *expected usage count*.

BP with a Depth limit (BPD) algorithm. The BPD algorithm [?] was proposed for optimizing the `XOR` count in depth-constrained circuits. The BPD used the following Thm. ?? to obtain the minimum depth to implement the target nodes.

Theorem 1 ([?]). *Let ~~$S = \{v_0, \cdots, v_{t-1}\}$~~ $S = \{v_0, \ldots, v_{t-1}\}$ be a set of nodes with $\mathcal{D}(v_i) = d_i$, then the lower bound of the depth of the circuit implementing the node ~~$z = v_0 \oplus \cdots \oplus v_{t-1}$ is $\lceil log_2 \sum_i^{t-1} 2^{d_i} \rceil$~~ $z = v_0 \oplus \cdots \oplus v_{t-1}$ is $\lceil \log_2 \sum_{i=0}^{t-1} 2^{d_i} \rceil$, and the lower bound is always achievable.*

Based on ~~this theorem, for each linear representation of a target vector $\mathbf{t}$, its lower bound of the depth can be estimated.~~

~~(~~Thm. ??, they define the H-~~depth linear representation)Given a target vector $\mathbf{t}$ and a set of vectors B where each vector $\mathbf{b} \in B$ has a depth $\mathcal{D}(\mathbf{b})$, the set of all linear~~

~~representations of $\mathbf{t}$ with respect to B is defined as: $\mathrm{LinSet}(B, \mathbf{t}) = \{L \mid \mathbf{t} = \bigoplus_{\mathbf{b} \in L} \mathbf{b},\ L \subseteq B\}$.~~

~~For any linear representation $L \in \mathrm{LinSet}(B, \mathbf{t})$, the minimum depth required to compute $\mathbf{t}$ from L is $\mathcal{D}_{\min}(\mathbf{t}, L)$. Given a positive integer H, we define the H-depth linear representation satisfying the depth constraint as:~~

$$\mathrm{LinSet}_H(B, \mathbf{t}) = \{L \mid L \in \mathrm{LinSet}(B, \mathbf{t}), \mathcal{D}_{\min}(\mathbf{t}, L) \leq H\}.$$

~~The **H-distance** from B to $\mathbf{t}$ is defined as the minimal number of additional XOR gates~~ -distance, which represents the number of `XOR` operations required to implement ~~$\mathbf{t}$ without exceeding~~ the node without exceeding the depth ~~H, which is the minimal value of $|L| - 1$ over all linear representations satisfying the depth constraint:~~

$$\delta_H(B, \mathbf{t}) = \min\{|L| - 1 \mid L \in \mathrm{LinSet}_H(B, \mathbf{t})\}.$$

~~If no such linear representation exists, $\delta_H(B, \mathbf{t}) = \infty$.~~

~~The BPD~~ limit H. The only difference between BPD algorithm and BP algorithm ~~utilizes $\delta_H(B, \mathbf{t})$ to construct the distance vector $Dist$ and total distance $TotalDist$, then selects pairs following the strategy of the original BP algorithm .~~

~~**The extended BPD algorithm**~~ is the distance chosen when computing the objective function.

~~Building upon the BPD algorithm,~~

Extended-BP (EBP) algorithm. Jeon et al.~~proposed the extended BPD (eBPD) algorithm [?], which provides a solution framework for the DC-eSLP problem. The eBPD (Extended Boyar–Peralta with Depth limit) algorithm is a heuristic method designed to optimize~~ [?] proposed the Extended-BP algorithm to solve the S-box circuits ~~by minimizing the number of XOR gateswhile respecting a predefined depth constraint. It extends the original Boyar–Peralta (BP) algorithm to handle circuits containing nonlinear gates (e.g., AND, OR), which are common in S-box implementations.~~

~~In the eBPD algorithm , for each target vector $\mathbf{t}$, if there exists an H-depth linear combination of $\mathbf{t}$ with respect to the current set of vectors, the distance is computed based on Definition ??. If no such H-depth linear combination exists, it means that generating $\mathbf{t}$ requires some vector $\mathbf{g}_i$ that has not been activated (i.e.,~~ minimization with fixed nonlinear gates. Since the inputs and outputs of nonlinear gates are fixed, the extended-BP algorithm adds the input nodes of nonlinear gates to the set of targets and adds the output node of that gate to S if two inputs of a nonlinear gate are implemented. That is, if $\mathbf{r}_{2i}$ ~~or~~ and $\mathbf{r}_{2i+1}$ ~~has not been generated), then the distance is set to ∞. When calculating the reduction in $TotalDist$, the authors adjust the distance metric by excluding values that are ∞.~~ belong to S, $\mathbf{g}_i$ is added to S, and this process is a new "pre-emptive" strategy.

We use Exa. ?? to illustrate the computation process of the LCB-BP and Extend-BP algorithms under no depth constraints. Tab. ?? is the computation process for step 1-6 in Exa. ??. Without depth constraints, only vectors linearly expressible by available nodes are considered in computing minimal linear representations. At step 1, since $\mathbf{g}_1$ is not yet generated, there is no linear representation for $\mathbf{r}_6$. Thus, only representations for $\mathbf{r}_0$ through $\mathbf{r}_5$ are listed in Tab. ??. Whitout a depth limit, every pair in any minimal linear representation is a candidate. For example, in the first step, all pairs $\{\mathbf{e}_i, \mathbf{e}_j\}$ with $0 \leq i < j \leq 5$ are considered, yielding 15 candidate pairs. The pair with the largest *expected usage count* is selected, which is $\{\mathbf{e}_0, \mathbf{e}_1\}$.

2.2 ~~The local optimization methods~~

~~In [?], two local optimization techniques were proposed. One technique represents an implementation as a tree structure, where each XOR operation is encoded in a binary tree containing three nodes.An exhaustive search is then performed on a partial section of the tree. However, since the tree representation of a matrix implementation is typically too large to permit an exhaustive search over the entire structure, this approach is necessarily limited in its scope. The other technique, called swapping orders, identifies and reorders operations within specific segments of an implementation to eliminate redundant operations and reduce gate depth.~~

3 The General-BP framework for depth-constrained circuit minimization

~~Subsequently, [?] proposed 11 reduction rules to eliminate redundant operations in sequence.They claimed that Rule 3 represents a variant of the swapping orders technique from [?], but is more general in its application.~~

~~In [?], a graph-based representation was adopted for sequences, along with a novel strategy for selecting subsequences for local optimization . The authors observed that subgraphs consisting of nodes with out-degree 1 are more amenable to replacement using fewer nodes. Therefore, they selectively optimized only those subsets of nodes in the graph representation that have an out-degree of 1. Moreover, they identified that the optimization problem could also be formulated as a Straight-Line Program (SLP)problem, thereby enabling the application of the BP algorithm for effective local optimization.~~ In Sec. ??, we identified four aspects of the original BP that can be further improved. In Sec. ??, we provide a detailed analysis of the causes of these limitations and outline the corresponding improvement strategies. To incorporate these improvements into the BP framework, Sec. ?? introduces a unified architecture capable of addressing both depth-constrained linear-layer optimization and S-box circuit optimization with fixed nonlinear gates. The four specific improvements are then presented in Sec. ?? through Sec. ??.

4 ~~An approach for optimizing the depth-constrained circuit~~

~~In this section, we improve the approaches for~~

3.1 Four limitations and motivations for improvements

Limitation 1: The low efficiency of depth-constrained circuit optimization limits the number of algorithm runs that can be performed within a practical timeframe. In the original BP algorithm, given a current node set S (including inputs X), the candidate set is $\{\{\mathbf{w}_i, \mathbf{w}_j\} \mid \mathbf{w}_i, \mathbf{w}_j \in S, \mathbf{w}_i \neq \mathbf{w}_j\}$. With l nodes, this yields $l(l-1)/2$ candidates. At each step, it computes $totalDist(S_{ij}, Y)$ for every candidate and selects the best, resulting in $O(l^2)$ complexity per step. LCB-BP improves efficiency by storing, for each target, its minimal linear representations with respect to S. However, LCB-BP cannot be directly applied to depth-constrained ~~circuit optimization~~problems for two reasons. First, ~~we introduce the core optimization algorithms. Secondly, we present a novel strategy for target vector selection and two tie-breaking criteria to enhance algorithmic performance. Finally, we develop a new circuit representation method.~~ it does not account for depth when computing linear representations. For instance, if we set the depth limit of $\mathbf{r}_1$ and $\mathbf{r}_4$ to 3, in step 4 of Tab. ??, the representation $\mathbf{r}_1 = \mathbf{w}_2 \oplus \mathbf{e}_1$ gives depth 4, violating a depth limit of 3. Second, without a depth constraint, every pair in a minimal representation

is considered a valid candidate. Under a depth limit, careful selection is needed. For example, in step 3, $\mathbf{r}_4 = \mathbf{w}_1 \oplus \mathbf{e}_3 \oplus \mathbf{e}_4$ contains three candidate pairs, but only $\{\mathbf{e}_3, \mathbf{e}_4\}$ is valid.

3.2 ~~A new algorithm for generating the depth-constrained circuit~~

Motivations 1. To address this limitation, we define the *depth-constrained minimal linear representation* (Def. **??**), which explicitly enforces that the depth lower bound of the representation does not exceed the given limit. Moreover, even when such a representation exists, not every pair of vectors appearing in it can be safely used to generate a new intermediate node: some pairs, when replaced by their sum, would cause the depth lower bound of the resulting representation to violate the depth limit. This observation leads to the notion of a *valid pair*. The valid pair is a pair that preserves depth feasibility. By storing these representations and valid pairs, the algorithm provides a solid foundation for the subsequent steps of the General-BP framework, enabling it to respect depth constraints while avoiding redundant computations.

Limitation 2: The fixed depth-limit setting on inputs of nonlinear gates restricts the number of candidate pairs. In S-box optimization with fixed nonlinear gates, depth bounds must be assigned to the input vectors of these gates. Let H be the global depth limit. Each $\mathbf{r}_i \in R$ receives a depth bound H_i, which should be maximized subject to the condition that if all $\mathbf{r}_i$ meet their bounds, the estimated depths (via Thm. **??**) of all target outputs remain within H. Depth bound assignments are not unique, and different choices lead to different optimization results. The existing method [**?**] uses a fixed depth-limit setting. If the initial bounds are poorly chosen, the algorithm may fail to produce sufficiently good results.

Table 2: The node sequence of vectors in $R \cup Y$

Vector	Node Sequence	Vector	Node Sequence
$\mathbf{r}_0$	$\vec{S}(\mathbf{r}_0)=[\mathbf{w}_0, \mathbf{w}_1]$	$\mathbf{r}_8$	$\vec{S}(\mathbf{r}_8)=[\mathbf{w}_8, \mathbf{w}_9, \mathbf{w}_{12}, \mathbf{w}_{14}, \mathbf{w}_{19}, \mathbf{w}_{20}]$
$\mathbf{r}_1$	$\vec{S}(\mathbf{r}_1)=[\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_3]$	$\mathbf{r}_9$	$\vec{S}(\mathbf{r}_9)=[\mathbf{w}_5, \mathbf{w}_9, \mathbf{w}_{13}, \mathbf{w}_{15}, \mathbf{w}_{16}]$
$\mathbf{r}_2$	$\vec{S}(\mathbf{r}_2)=[\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2]$	$\mathbf{r}_{10}$	$\vec{S}(\mathbf{r}_{10})=[\mathbf{w}_5, \mathbf{w}_9, \mathbf{w}_{13}, \mathbf{w}_{15}, \mathbf{w}_{17}]$
$\mathbf{r}_3$	$\vec{S}(\mathbf{r}_3)=[\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_4, \mathbf{w}_6]$	$\mathbf{r}_{11}$	$\vec{S}(\mathbf{r}_{11})=[\mathbf{w}_8, \mathbf{w}_9, \mathbf{w}_{12}, \mathbf{w}_{13}, \mathbf{w}_{15}, \mathbf{w}_{17}, \mathbf{w}_{18}, \mathbf{w}_{23}, \mathbf{w}_{24}]$
$\mathbf{r}_4$	$\vec{S}(\mathbf{r}_4)=[\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_4]$	$\mathbf{y}_0$	$\vec{S}(\mathbf{y}_0)=[\mathbf{w}_5, \mathbf{w}_9, \mathbf{w}_{10}, \mathbf{w}_{21}, \mathbf{w}_{25}, \mathbf{w}_{26}, \mathbf{w}_{27}, \mathbf{w}_{28}, \mathbf{w}_{29}]$
$\mathbf{r}_5$	$\vec{S}(\mathbf{r}_5)=[\mathbf{w}_0, \mathbf{w}_1, \mathbf{w}_2, \mathbf{w}_4, \mathbf{w}_7]$	$\mathbf{y}_1$	$\vec{S}(\mathbf{y}_1)=[\mathbf{w}_8, \mathbf{w}_{11}, \mathbf{w}_{21}, \mathbf{w}_{22}]$
$\mathbf{r}_6$	$\vec{S}(\mathbf{r}_6)=[\mathbf{w}_8, \mathbf{w}_{11}]$	$\mathbf{y}_2$	$\vec{S}(\mathbf{y}_2)=[\mathbf{w}_5, \mathbf{w}_9, \mathbf{w}_{12}, \mathbf{w}_{31}, \mathbf{w}_{32}]$
$\mathbf{r}_7$	$\vec{S}(\mathbf{r}_7)=[\mathbf{w}_5, \mathbf{w}_{10}]$	$\mathbf{y}_3$	$\vec{S}(\mathbf{y}_3)=[\mathbf{w}_8, \mathbf{w}_{21}, \mathbf{w}_{33}, \mathbf{w}_{34}, \mathbf{w}_{35}]$

Motivation 2. When a generated vector $\mathbf{r}_i$ has actual depth $\mathcal{D}(\mathbf{r}_i) < H_i$, the unused depth budget can be redistributed to increase the bounds of remaining ungenerated $\mathbf{r}_j$, potentially yielding better results. This observation motivates a dynamic depth-limit allocation. We propose two improvements in Sec. **??**: first, a new ordering for the initial bound assignment; second, dynamic depth-limit allocation that updates the depth bounds during the computation. This combined approach enables the algorithm to produce improved optimization results.

Limitation 3: Inaccurate candidate evaluation score misleads the selection on new vectors. In the LCB-BP algorithm, the candidate evaluation score typically selects the pair with the highest frequency. For each pair, this frequency serves as an estimate of the pair's actual contribution; we refer to this as the expected usage count, which can be computed after the full sequence is constructed. The *actual usage count* $\mathcal{U}_A(\mathbf{w}_i)$ is then defined as the number of vectors $\mathbf{t} \in R \cup Y$ such that $\mathbf{w}_i$ belongs to $\vec{S}(\mathbf{t})$. For each target

vector $\mathbf{t} \in R \cup Y$, its node sequence $\vec{S}(\mathbf{t})$ captures only the linear operations involved in its generation-i.e., the `XOR` steps that directly contribute to producing $\mathbf{t}$ from previously available vectors, excluding steps of generating fixed nonlinear gates outputs. Tab. **??** lists the node sequence of target vectors in Exa. **??**. For example, $\mathcal{U}_A(\mathbf{w}_{14}) = 1$ because it appears only in $\vec{S}(\mathbf{r}_8)$. However, during step 15 in Tab. **??**, its expected usage count was 2, since it appeared in the minimal linear representations of both $\mathbf{r}_8$ and $\mathbf{r}_{11}$. A discrepancy between expected and actual usage arises due to the evolution of minimal linear representations during the algorithm.

To understand this, we first clarify two types of minimal representations:

Inherited minimal linear representations: If the pair selected in iteration $k+1$ appears in a minimal representation L_k of $\mathbf{y}$ with respect to S, then we can directly construct a new minimal representation L_{k+1} for $\mathbf{y}$ with respect to $S_{ij} = S \cup \{\mathbf{w}_i \oplus \mathbf{w}_j\}$ by replacing $\{\mathbf{w}_i, \mathbf{w}_j\}$ with their sum. The length of L_{k+1} is $|L_k| - 1$, so the distance decreases by 1.

Non-inherited minimal linear representations: If the selected pair does not appear in any minimal representation of $\mathbf{y}$ but does appear in a longer representation L_k^* of length $|L_k| + 1$, then we can obtain a new representation L_{k+1}^* of length $|L_k|$. The distance to $\mathbf{y}$ remains unchanged, but a new minimal representation appears.

The reason for discrepancy between expected and actual usage count of $\mathbf{w}_{14}$ is as follows. At step 15, the node $\mathbf{w}_{14}$ contributed to an inherited minimal representation of $\mathbf{r}_{11}$. Later, at step 17, a non-inherited representation of $\mathbf{r}_{11}$ emerged, and $\mathbf{r}_{11}$ was ultimately generated using that new representation, which did not involve $\mathbf{w}_{14}$. Consequently, node $\mathbf{w}_{14}$ never actually contributed to $\mathbf{r}_{11}$, despite being counted in the expected usage. This mismatch between the projected and realized contribution exemplifies the inaccurate candidate evaluation score problem in greedy selection.

Motivation 3. To mitigate this problem, we refine the scoring rule by incorporating the observation that targets with longer minimal representations are more prone to developing new non-inherited representations, and thus are more likely to cause mismatches. Accordingly, we assign weight 1 only to targets whose minimal representation length does not exceed a threshold ℓ; longer representations receive weight 0, thereby reducing their misleading influence. In addition, when a target admits a minimal representation of length 2, the corresponding pair is guaranteed to appear in that target's node subsequence, so we give it infinite priority. This weighted scoring strategy aims to align the estimated score more closely with the actual usage, improving the results of BP algorithm.

Algorithm 1: The basic framework for depth-constrained circuits minimization

Input : $H \in \mathbb{N}$: the depth limit of the circuit; $X \subseteq \mathbb{F}_2^n$: the set of initial input vectors; $R \subseteq \mathbb{F}_2^n$: the input vectors of nonlinear operations; $G \subseteq \mathbb{F}_2^n$: output vectors of nonlinear operations; $Y \subseteq \mathbb{F}_2^n$: target vectors.

Output : $\vec{S}$: node sequence; $\vec{S}_{\mathtt{OP}}$: operation sequence.

```
begin
    T ← Y ∪ R ∪ G; S ← X
    S⃗ ← empty sequence; S⃗_OP ← empty sequence; h ← 0
    while T ⊈ S do
        h ← AssignDepthLimit(T, S, H, h)
        R_min, P_valid ← GenRepAndPairs(T, S, h)
        Score ← ComputeScore(R_min, P_valid)
        S, S⃗, S⃗_OP ← AppendNode(S, S⃗, S⃗_OP, Score)
    S⃗, S⃗_OP ← SubcircuitReconstruct(S⃗, S⃗_OP)
    return S⃗, S⃗_OP
```

Limitation 4: Indistinguishable candidate evaluation score misleads the selection on new vectors. In the later stages of the BP algorithm, it often happens that all remaining

candidate pairs have the same expected usage count (typically 1). Yet different choices can lead to vastly different outcomes.

For example, at step 27 in Tab. **??**, only $\mathbf{y}_0, \mathbf{y}_2, \mathbf{y}_3$ remain, with minimal representations: $\mathbf{y}_0 = \mathbf{e}_4 \oplus \mathbf{w}_{10} \oplus \mathbf{g}_2 \oplus \mathbf{g}_4 \oplus \mathbf{g}_5$, $\mathbf{y}_2 = \mathbf{e}_2 \oplus \mathbf{g}_0 \oplus \mathbf{g}_2 \oplus \mathbf{g}_3$, $\mathbf{y}_3 = \mathbf{e}_1 \oplus \mathbf{e}_6 \oplus \mathbf{g}_1 \oplus \mathbf{g}_4$. All candidate pairs have expected usage count 1. Following the original order (Tab. **??**) keeps every score at 1. If instead we generate $\mathbf{w}'_{26} := \mathbf{e}_6 \oplus \mathbf{g}_4$, a new non-inherited representation appears for $\mathbf{y}_0$: $\mathbf{y}_0 = \mathbf{e}_4 \oplus \mathbf{w}'_{26} \oplus \mathbf{g}_0 \oplus \mathbf{g}_2 \oplus \mathbf{g}_5$. Then the pair $\{\mathbf{g}_0, \mathbf{g}_2\}$ now appears in the representations of both $\mathbf{y}_0$ and $\mathbf{y}_2$, raising its expected usage count to 2 and enabling better reuse. Although $\{\mathbf{e}_6, \mathbf{g}_4\}$ and $\{\mathbf{w}_{10}, \mathbf{g}_2\}$ have the same evaluation score, the former leads to a better outcome. Selecting the right pair in such a situation is hard, but crucial for achieving high-quality results.

Motivation 4. To address this limitation, we propose a subcircuit reconstruction technique. The key idea is to identify subcircuits that were generated under such indistinguishable scoring conditions and attempt to replace them with more efficient ones. By treating a selected set C of vectors as a subcircuit to be redesigned, we isolate a local reconstruction problem: find a new set C' of vectors that produces the same outputs using fewer operations, while preserving the functionality and depth constraints of the surrounding circuit. This approach effectively post-processes the parts of the circuit where the greedy selection was most likely to have made a poor decision, thereby improving the final result without exhaustively exploring all alternatives during construction.

3.2 The basic framework for depth-constrained circuit minimization

~~Building upon the idea of computing linear combinations of target vectors from [?],~~ Since the optimization problem for S-box circuits, after fixing the nonlinear operations, reduces to optimizing the number of `XOR` operations, it becomes essentially the same as linear-layer circuit optimization. Both can be addressed using the BP algorithm. To provide a unified solution for `XOR` count optimization in both fixed S-box and linear-layer circuits, we introduce the following framework.

In an S-box circuit, once the input and output of each nonlinear gate are fixed, these nodes can be treated as additional target vectors. Thus, the overall target set becomes $T = Y \cup G \cup R$. In the linear-layer case, both R and G are empty, and the target set reduces to $T = Y$. The original notion of distance is no longer suitable for problems involving nonlinear operations. To unify the optimization of S-box and linear-layer circuits, we adopt the number of operations to replace the distance. For a target vector $\mathbf{t}$, and a current set of available vectors S, the *operation count* is defined as the minimum number of operations required to generate $\mathbf{t}$ from vectors in S, where an operation can be either a linear `XOR` or a nonlinear gate $\mathtt{NL}_i$. The algorithm terminates when the sum of operation counts over all target vectors reaches zero.

~~we propose a novel algorithm for computing the~~ The basic framework consists of two parts: generating an initial sequence and performing subcircuit reconstruction. Here are the core steps to generate the initial sequences.

1. `GenRepAndPairs` : compute the depth-constrained ~~.~~ minimal-operation representation of each target and select the valid pairs to construct the candidates.
2. `AssignDepthLimit` : assign depth limits to the vectors in R and G dynamically.
3. `ComputeScore` : evaluate each candidate using a scoring function and output the pair with the highest score.
4. Select the candidate with the highest score, and append the newly generated vector along with its operation to the current set, node sequence, and operation sequence.

After generating the initial sequence, General-BP further optimizes the result via the `SubcircuitReconstruct` function. The basic framework is illustrated in Alg. **??**. ~~This algorithm achieves substantial efficiency improvements over the original BPD algorithm.~~

~~Unlike the exhaustive pairwise evaluation approach in BPD algorithm, our method identifies candidate vector pairsthat can effectively reduce distances while respecting depth constraints. Our algorithm extends the LCB-BP algorithm by incorporating depth constraints at each step, ensuring that the selected pairs not only minimize the distance but also satisfy the given depth limitations throughout the sequence construction process.~~

~~We now present several key definitions that form the foundation of our algorithm. (The H-depth minimal linear representation)~~

3.3 Generation of minimal-operation representations and candidate pairs under a depth limit

A minimal-operation representation may include fixed nonlinear operations. However, generating such a representation that respects a given depth limit is difficult. Thus, we adopt an incremental strategy as follows: at each step, we first check whether the necessary output vectors for the nonlinear gates have been generated. Once available, we rewrite the nonlinear expression for the target vector as a linear expression using these existing vectors. If such a linear expression is found, we then compute a minimal linear representation that meets the depth constraint. Such a representation is defined as a depth-constrained minimal linear representation.

Definition 1 (The depth-constrained minimal linear representation)**.** Given a vector $\mathbf{t}$, a set of vectors S where each $\mathbf{w} \in S$ has a depth $\mathcal{D}(\mathbf{w})$, and a positive integer $H \in \mathbb{N}$. A linear representation L is defined as a ~~set of vectors B where each vector $\mathbf{b} \in B$ has a depth $\mathcal{D}(\mathbf{b})$, and a positive integer H, L is defined as the H-*depth minimal linear representation* of~~ *depth-constrained minimal linear representation* of $\mathbf{t}$ with respect to ~~B~~ S if ~~$|L| = \min\{|L| \mid L \in \mathrm{LinSet}_H(B, \mathbf{t})\}$ where $\mathrm{LinSet}_H(B, \mathbf{t})$ is the H-depth linear representation . The set of all H-depth minimal linear representations is then denoted by $\mathrm{MinLinSet}_H(B, \mathbf{t})$.~~ $|L| = \min\{|L| \mid L \subseteq S \text{ and } \bigoplus_{\mathbf{v} \in L} \mathbf{v} = \mathbf{t} \text{ and } \lceil \log_2 \sum_{\mathbf{v} \in L} 2^{\mathcal{D}(\mathbf{v})} \rceil \leq H\}$.

~~(H-feasible pair) Given a vector $\mathbf{t}$, a set of vectors B where each vector $\mathbf{b} \in B$ has a depth $\mathcal{D}(\mathbf{b})$, and a positive integer H, let L be an H-depth~~

For example, when setting the depth limit of $\mathbf{r}_1$ to be 3, then the depth-constrained minimal linear representation of ~~$\mathbf{t}$ with respect to B.A pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ is called an H-*feasible pair* of L if $\mathcal{D}_{\min}(\mathbf{t}, L') \leq H$, where $L' = (L \setminus \{\mathbf{b}_i, \mathbf{b}_j\}) \cup \{\mathbf{b}_i \oplus \mathbf{b}_j\}$ and $\mathbf{b}_i, \mathbf{b}_j \in L$. (The~~ $\mathbf{r}_1$ is $\mathbf{e}_0 \oplus \mathbf{e}_2 \oplus \mathbf{e}_3$ instead of $\mathbf{w}_2 \oplus \mathbf{e}_1$, since the lower bound of depth with respect to $\mathbf{w}_2 \oplus \mathbf{e}_1$ is $\lceil \log_2(2^3 + 2^0) \rceil = 4$. The lower bound of depth with respect to $\mathbf{e}_0 \oplus \mathbf{e}_2 \oplus \mathbf{e}_3$ is $\lceil \log_2(2^0 + 2^0 + 2^0) \rceil = 2$, which satisfies the depth limit 3.

If the depth limit H ~~-depth Support) Given a target vector $\mathbf{t}$, a set of vectors B where each vector $\mathbf{b} \in B$ has a depth $\mathcal{D}(\mathbf{b})$, and a positive integer H, the H-*depth support* of~~ for a target vector $\mathbf{t}$ ~~with respect to B is defined as $\mathrm{support}_H(B, \mathbf{t}) = \{\{\mathbf{b}_i, \mathbf{b}_j\} \mid \exists L \in \mathrm{MinLinSet}_H(B, \mathbf{t})$ s.t. $\{\mathbf{b}_i, \mathbf{b}_j\}$ is an H-feasible pair of $L\}$. Here we use the following example to illustrate the definitions.~~ is set too low, it is possible that $L_H(S, \mathbf{t}) = \emptyset$, meaning no minimal linear representation meets the depth constraint. As the set S expands, such a representation may eventually become possible. If $\mathbf{t}$ is the only unresolved target and still cannot be generated, then the sequence cannot be completed. However, these situations are relatively rare.

~~Consider target vector $\mathbf{t}_1$ in Example ?? with depth bound $H = 4$.The vector $\mathbf{t}_1$ has a linear representation $L = \{\mathbf{b}_2, \mathbf{b}_9, \mathbf{b}_5, \mathbf{b}_3\}$ with respect to $B = \{\mathbf{b}_0, \cdots, \mathbf{b}_9\}$ The depth bound calculation gives:~~

$$\mathcal{D}_{\min}(\mathbf{t}_1, L) = \lceil \log_2(2^{\mathcal{D}(\mathbf{b}_2)} + 2^{\mathcal{D}(\mathbf{b}_9)} + 2^{\mathcal{D}(\mathbf{b}_5)} + 2^{\mathcal{D}(\mathbf{b}_3)}) \rceil = \lceil \log_2(10) \rceil = 4 \leq H.$$

~~This representation uses 4 vectors, and there is no representation with fewer vectors that satisfies the depth constraint, so $L \in \mathrm{MinLinSet}_H(B, \mathbf{t}_1)$.~~ To uniformly handle linear operations and fixed nonlinear operations, two types of minimal-operation representations are considered in General-BP:

~~Next, consider the pair $\{\mathbf{b}_2, \mathbf{b}_9\}$. If we replace this pair with their XOR $\mathbf{b}_{10} = \mathbf{b}_2 \oplus \mathbf{b}_9$, we get $L' = \{\mathbf{b}_{10}, \mathbf{b}_5, \mathbf{b}_3\}$. The depth bound of L' is:~~

$$\mathcal{D}_{\min}(\mathbf{t}_1, L') = \lceil \log_2(2^{\mathcal{D}(\mathbf{b}_{10})} + 2^{\mathcal{D}(\mathbf{b}_5)} + 2^{\mathcal{D}(\mathbf{b}_3)}) \rceil = \lceil \log_2(2^2 + 2^2 + 2^1) \rceil = \lceil \log_2(10) \rceil = 4 \leq H.$$

~~Therefore, $\{\mathbf{b}_2, \mathbf{b}_9\}$ is an H-feasible pair of L~~1. *Depth-constrained minimal linear representations.* For each target that can be linearly expressed by vectors in S, we compute its depth-constrained linear representation.

~~For the same target vector $\mathbf{t}_1$, we can identify multiple H-feasible pairs in its minimal linear representations. Thus, the H-depth support is:~~

$$\mathrm{support}_H(B, \mathbf{t}_1) = \{\{\mathbf{b}_2, \mathbf{b}_9\}, \{\mathbf{b}_2, \mathbf{b}_3\}, \{\mathbf{b}_2, \mathbf{b}_5\}, \{\mathbf{b}_9, \mathbf{b}_5\}, \{\mathbf{b}_9, \mathbf{b}_3\}, \{\mathbf{b}_5, \mathbf{b}_3\}\}.$$

~~In [?], Lemma 1 states that without depth constraints, for any target~~ 2. *Representations of nonlinear operations.* For a vector ~~$\mathbf{t}$, if any pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ from the support of $\mathbf{g}_i$~~ whose two input variables $\mathbf{r}_{2i}$ and $\mathbf{r}_{2i+1}$ are both in S, its minimal representation is given by $\mathtt{NL}_i(\mathbf{r}_{2i}, \mathbf{r}_{2i+1})$. Furthermore, we also express this representation as $\{\mathbf{r}_{2i}, \mathbf{r}_{2i+1}\}$, allowing it to be treated uniformly as a "pre-emptive" case.

The set of all depth-constrained minimal-operation representations of $\mathbf{t}$ ~~is used to generate a new vector $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$ and added to the sequence, the distance of $\mathbf{t}$ will decrease by 1. This property also holds in the~~ with respect to set S is denoted by $\mathcal{R}_H(S, \mathbf{t})$.

In the depth-constrained ~~scenario discussed in this section: adding any new vector generated from an H-feasible pair will reduce the H-distance of the corresponding target vector by 1. Therefore, we define the *H-depth frequency* to measure the utility of pairs under depth constraints, and at each step, we select the pair with the highest H-depth frequency to add to the sequence.~~circuit minimization problem, the construction of candidate pairs differs from that in [?]: not every pair appearing in a depth-constrained minimal linear representation is considered valid. For a given representation L and a pair $\{\mathbf{w}_i, \mathbf{w}_j\} \in L$, the pair is valid with respect to L only if the depth lower bound of the new linear representation $L' = (L \setminus \{\mathbf{w}_i, \mathbf{w}_j\}) \cup \{\mathbf{w}_i \oplus \mathbf{w}_j\}$ also satisfies the depth limit H, i.e., $\mathcal{D}_{\min}(L') \leq H$, where $\mathcal{D}_{\min}(L')$ is $\lceil \log_2 \sum_{\mathbf{w} \in L'} 2^{\mathcal{D}(\mathbf{w})} \rceil$ according to Thm. ??. A pair is invalid with respect to a given representation if it does not satisfy the depth limit.

~~(H-depth)Given a set of vectors B and an output target set T, let H_i be the depth bound of the $\mathbf{t}_i \in T$.~~For example, in step 3 of Tab. ??, set the depth limit of $\mathbf{r}_4$ to 3. The ~~*H-depth frequency* of a pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ is defined as $\mathrm{frequency}_{B,T}(\mathbf{b}_i, \mathbf{b}_j) = |\{\mathbf{t}_i \mid \{\mathbf{b}_i, \mathbf{b}_j\} \in \mathrm{support}_{H_i}(B, \mathbf{t}_i), \mathbf{t}_i \in T\}|$~~

~~We present Algorithm ?? to generate the~~ depth-constrained ~~extended with the same effectiveness as the cBPD algorithm [?], but achieves significantly higher computational efficiency. The function $\mathtt{genDepthMinLinCombs}(B, \mathbf{t}, H)$ formulates and solves a Mixed Integer Linear Programming (MILP) model to generate all H-depth minimal linear representations of target vector $\mathbf{t}$ with respect to the vector set $\mathcal{B}$. This function ensures that all generated linear combinations satisfy both the depth constraint H and the minimality condition in terms of vector count. The complete pseudocode for this MILP-based procedure is provided in Appendix ??.In this algorithm, if a non-linear gate vector $\mathbf{g}_i$ can be activated (i.e., its activation conditions are met), we add $\mathbf{g}_i$ to the sequence. If $G^{(u)}$ and $R^{(2u)}$ are empty, our algorithm can solve the DC-SLP problem.~~ minimal linear representation of $\mathbf{r}_4$ remains $\mathbf{w}_1 \oplus \mathbf{e}_3 \oplus \mathbf{e}_4$. However, among the pairs in $\{\mathbf{w}_1, \mathbf{e}_3, \mathbf{e}_4\}$, only $\{\mathbf{e}_3, \mathbf{e}_4\}$ is valid according to the above definition of valid pairs.

Function 2: The `GenRepAndPairs` function

Input : $T \subseteq \mathbb{F}_2^n$: set of target vectors; $S \subseteq \mathbb{F}_2^n$: current vector set; $\mathbf{h} \in \mathbb{N}^{|T|}$: depth bounds for target vectors.

Output : $\mathcal{R}_H$: minimal-operation representations set; $\mathcal{P}_{\texttt{valid}}$: valid pair set.

```
1  begin
2      R_H ← an empty dictionary; P_valid ← ∅
3      for t_i ∈ T do
4          if t_i = g_j ∈ G and r_{2j}, r_{2j+1} are generated then
5              R_min(t_i) = {{r_{2j}, r_{2j+1}}}
6          else if t_i is a linear combination of vectors in S then
7              R_min(t_i) ← all minimal linear representation of t_i under the depth limit h_i.
8          else
9              R_min(t_i) ← ∅
10         R_H[t_i] = R_min(t_i)          // Store the mapping from t to its representations
11         for L ∈ R_min(t_i) do
12             for each pair {w_i, w_j} ⊆ L do
13                 if {w_i, w_j} is a valid pair then
14                     P_valid ← P_valid ∪ {{w_i, w_j}}
15     return R_H, P_valid
16 ~~The algorithm for genenrating the depth-constrained extended~~
```

3.4 ~~New heuristic strategies for selecting target vectors and pairs~~

~~**The strategy for selecting targets vectors** We observe that in certain cases, excluding some target vectors when counting pair frequencies can lead to better solutions. For example, consider a scenario where at iteration 10, a target vector $\mathbf{t}$ has two~~ The function `GenRepAndPairs` takes as input a target set T, a current vector set S, and depth limits for targets $\mathbf{h}$, where h_i denotes the i-th element of $\mathbf{h}$. For each target $\mathbf{t}_i \in T$ that is a combination of vectors in S, it computes all minimal linear representations $\mathcal{R}_{\min}(\mathbf{t}_i)$ under the depth limit h_i. If a target $\mathbf{t}_i = \mathbf{g}_j \in G$ and its input vectors $\mathbf{r}_{2j}, \mathbf{r}_{2j+1}$ are already generated, we regard $\{\mathbf{r}_{2j}, \mathbf{r}_{2j+1}\}$ as the minimal representation ~~$\mathbf{t} = \mathbf{b}_0 \oplus \mathbf{b}_8 \oplus \mathbf{b}_9 \oplus \mathbf{b}_{10} \oplus \mathbf{b}_{11}$ and $\mathbf{t} = \mathbf{b}_1 \oplus \mathbf{b}_2 \oplus \mathbf{b}_3 \oplus \mathbf{b}_4 \oplus \mathbf{b}_{12} \oplus \mathbf{b}_{13}$. $\{\mathbf{b}_1, \mathbf{b}_8\}$ is one H-feasible pair with highest frequency. The algorithm selects this pair and generates $\mathbf{b}_{17} = \mathbf{b}_1 \oplus \mathbf{b}_8$. However, in subsequent iterations (11-15), new vectors $\mathbf{b}_{18}$ to $\mathbf{b}_{21}$ are generated through pairs $\{\mathbf{b}_1, \mathbf{b}_2\}$, $\{\mathbf{b}_3, \mathbf{b}_4\}$~~and stores the mapping $\mathcal{R}_H[\mathbf{t}_i] = \mathcal{R}_{\min}(\mathbf{t}_i)$, and collects into the set $\mathcal{P}_{\texttt{valid}}$, which every pair $\{\mathbf{w}_i, \mathbf{w}_j\}$ appearing in any such representation that is valid. It returns the map $\mathcal{R}_H$ and the valid pair set $\mathcal{P}_{\texttt{valid}}$.

3.4 Dynamic depth-limit allocation for vectors in T

Jeon et al. [**?**] proposed a method for assigning depth bounds to a set of target vectors, given a sorting rule and starting depth values. Based on this general method, we perform depth-bound assignment in two phases, both using the above general method but differing in starting values and sorting rules.

1. Initial depth bound assignment. At the start, each target receives a starting value equal to its depth lower bound estimated by Thm. **??**. Then targets are sorted in descending order of their operation count, giving priority to those with larger counts.

2. Dynamic depth bound adjustment. When a target vector $\mathbf{t}_i$ is actually generated and its realized depth $\mathcal{D}(\mathbf{t}_i)$ is strictly less than its currently assigned bound h_i, we trigger an adjustment for the remaining *not yet generated* targets. The current bounds of all targets are taken as starting values. For each target, those with at least one invalid pair are processed first. Among these, we sort by the number of valid pairs in ascending order,

so targets with fewer valid pairs are adjusted first. When all pairs are valid for a target, ~~$\{\mathbf{b}_{12}, \mathbf{b}_{13}\}$, and $\{\mathbf{b}_{18}, \mathbf{b}_{19}\}$. Ultimately, $\mathbf{t}$ is implemented as $\mathbf{t} = \mathbf{b}_{20} \oplus \mathbf{b}_{21}$ without using $\mathbf{b}_{17}$, leading a difference between predicted and actual vector usage.~~ we process these last, sorting them by operation count in descending order.

~~To generate the shortest possible sequence, we aim to maximize vector reuse. The difference between predicted and actual frequencies can lead to longer sequences. Therefore, to improve solution quality, we should avoid such situations by implementing an appropriate target vector selection strategy~~The rationale is that the number of valid pairs directly influences the expected usage of candidates: more valid pairs imply more alternative paths, which increases the probability of node reuse and ultimately leads to a shorter sequence. By reallocating the unused depth budget (when a vector is shallower than its bound) to targets that have fewer valid pairs, we increase their pool of valid pairs, thereby enhancing the potential for node reuse in later steps. This dynamic redistribution adapts the depth bounds to the actual progress of the construction and yields better optimization results than a fixed setting.

~~We think that the difference between predicted and actual frequencies arises because the minimal linear representations of target vectors may change as new vectors are continuously added to the sequence. For instance, after the i-th iteration, a new minimal linear representation may emerge for target vector~~

Function 3: The `AssignDepthLimit` function

Input : $T \subseteq \mathbb{F}_2^n$: target vector set; $S \subseteq \mathbb{F}_2^n$: current vector set; $H \in \mathbb{N}$: circuit depth limit; $\mathbf{h}' \in \mathbb{N}^{|T|}$: current depth bounds.
Output : $\mathbf{h}$: depth limits for all targets.

```
begin
    // Set starting values in initial stage
    if h' is 0 then
        for t_i ∈ T do
            h'_i ← D_min(t_i)          // D_min(t_i) is the lower bound estimated via Thm. ??
            set the i-th element of h' to h'_i
        set h from h' use the general method according to the initial sorting rule
    if the last vector t_s added in S is a target vector and D(t_s) < h'_s then
        set h from h' use the general method according to the dynamic sorting rule
    return h
```

3.5 Candidate scoring and selection

The scoring rule for each candidate pair is as follows:
1. If a target $\mathbf{t}$ ~~, and~~ admits a depth-constrained minimal representation of length 2, i.e., $\mathbf{t} = \mathtt{OP}(\mathbf{w}_i, \mathbf{w}_j)$, the pair $\{\mathbf{w}_i, \mathbf{w}_j\}$ receives score $+\infty$ and is selected immediately.
2. Otherwise, a pair that appears as a valid pair in the ~~final implementation of $\mathbf{t}$ might be based on this newly appeared representation. Therefore, when counting pair frequencies before the i-th iteration, pairs in the support of target vector $\mathbf{t}$ should not be considered~~ depth-constrained minimal representations of n targets receives score n.

~~However, it is currently impractical to determine which target vectors to filter in each iteration by comparing predicted and actual frequencies, because the actual frequency can only be known after the entire sequence is generated. Therefore, we need a heuristic strategy that can be implemented during the iterative process.~~

~~Given a target vector $\mathbf{t}$ and its current minimal linear representation L, we aim to develop a strategy that can predict whether~~ The score in item 2 is the *expected usage count* of the pair. However, this may differ from the *actual usage count* observed in the final sequence. A pair expected to be useful for a target $\mathbf{t}$ ~~is likely to develop a new minimal linear representation. This strategy primarily considers *the length of the minimal linear combination* and *the size of its support*.The length of the linear combination is considered~~

~~because if the current minimal representation is already short, the probability of an even shorter representation emerging is relatively low. Therefore, our selection criterion must account for the length of the linear combination.The size of the support is considered because, as mentioned in the subsection ??, pairs within the support of~~ may never appear in $\mathbf{t}$~~are precisely those that can reduce the distance to~~ 's node subsequence. This happens when: (i) at step k, the pair is selected from a representation L of $\mathbf{t}$~~, i.e., they can shorten the current minimal linear~~ ; (ii) later a new non-inherited representation ~~. Thus, if the number of feasible pairs is small, the likelihood of the current linear combination being shortened is reduced, increasing the probability that a new , shorter linear representation may appear.~~L' of $\mathbf{t}$ appears; (iii) all subsequent operations for $\mathbf{t}$ are taken from L', so the pair never contributes to $\mathbf{t}$.

~~In the absence of depth constraints, if $|L| = k$,~~ An example appears in Tab. ??: at step 15, pair $\{\mathbf{e}_5, \mathbf{w}_{12}\}$ is a valid pair in the ~~number of its feasible pairs is fixed at $k(k-1)/2$. Longer linear representations have a higher probability of being shortened. Under depth constraints, however, the number of its H-depth feasible pairs depends on the depth bound H. A lower depth bound H results in fewer H-depth feasible pairs, a smaller support, and a consequently higher probability of new minimal linear representations emerging.~~

~~Therefore, our strategy filters out those target vectors whose current minimal linear representations are relatively long and whose corresponding supports are relatively small.~~

~~The new tie-breaking strategy~~ ~~In [?], a tie-breaking strategy is proposed: if there is a tie between two or more candidate gates, the program selects the gate that maximizes the Euclidean norm of the distance vector $Dist$. We first explain the rationale of this strategy from the perspective of support sets. Then, for~~ representation of $\mathbf{r}_{11}$. The depth-constrained ~~problems, we propose a new tie-breaker based on support sets.~~ minimal linear representation of $\mathbf{r}_{11}$ is $\mathbf{e}_0 \oplus \mathbf{e}_5 \oplus \mathbf{w}_{13} \oplus \mathbf{w}_{11} \oplus \mathbf{w}_{12}$. Hence $\mathbf{w}_{14} := \mathbf{e}_5 \oplus \mathbf{w}_{12}$ was expected to help generate $\mathbf{r}_{11}$. But from step 17 a non-inherited representation of $\mathbf{r}_{11}$ emerges, and later steps never use that original representation. Consequently $\mathbf{w}_{14}$ does not belong to $\vec{S}(\mathbf{r}_{11})$ and its actual usage is zero, an overestimation.

~~The maximum norm criterion prioritizes reducing the distance of target vectors with shorter minimal linear representations. Under no depth constraints, we explain this as follows: a target vector with a shorter minimal linear representation usually has a smaller support set. Without knowing the final solution, each pair in the support has a probability of being the correct choice.If a target vector $\mathbf{t}$ has only one pair in its support, that pair must be the unique correct choice, which is the idea behind "pre-emptive" choices. When the support size is small, the probability that a pair is correct is high, and this probability decreases as the support size increases. Without depth constraints, distance and support size are positively correlated; thus, using the maximum norm criterion increases the likelihood of selecting correct pairs, generally leading to higher-quality solutions.~~

~~However, under depth constraints, the support size is influenced by both~~ Ideally, this problem could be avoided by estimating, for each $\mathbf{t}_k$, the probability $p_{i,j,k}$ that a pair $\{\mathbf{w}_i, \mathbf{w}_j\}$ will actually be used in the generation of $\mathbf{t}_k$. A weighted score could then be defined as $Score_{i,j} = \sum_{\mathbf{t}_k \in T} s_{i,j,k} \cdot w_{i,j}(\mathbf{t}_k)$, where $s_{i,j,k} = 1$ if $\{\mathbf{w}_i, \mathbf{w}_j\}$ is a valid pair for $\mathbf{t}_k$ (otherwise 0), and the ~~length of linear representation and the depth bound. Therefore, the maximum norm criterion may not be optimal.In [?], the authors mention that in their search for depth-minimal implementations of linear layer matrices, they use a tie-breaking rule that minimizes the Euclidean norm. This strategy can also be explained via support sets: since they set the depth bound to the minimum value , longer linear representations may have higher depth lower bounds and potentially fewer H-feasible pairs.Thus, by using the minimum norm criterion at the lowest depth bound, they select pairs with smaller support sets.~~ weight $w_{i,j}(\mathbf{t}_k)$ reflects the estimated probability $p_{i,j,k}$.

~~Based on this, we propose a new tie-breaking strategy called the average support minimizationcriterion. When a tie occurs between candidate pairs, for each~~ In practice, accurately computing these probabilities is infeasible. Nevertheless, we observe that targets with *longer minimal representations* are more prone to developing new non-inherited representations and therefore more likely to cause mismatches between expected and actual usage count. Based on this observation, we adopt a simple threshold-based heuristic: if the length of a target's minimal representation exceeds a predefined threshold ℓ, its weight is set to 0; otherwise, its weight is 1. This reduces the misleading influence of overly long representations during scoring. This perspective also justifies the first scoring rule. When a target ~~vector $\mathbf{t}_i$ affected by a candidate pair p, we define the average support size as: $\#avg_supp_i = \frac{|\text{support}_{H_i}(B,\mathbf{t}_i)|}{|\text{MinLinSet}_{H_i}(B,\mathbf{t}_i)|}$ where $|\text{MinLinSet}_{H_i}(B,\mathbf{t}_i)|$ is the number of minimal linear representations of $\mathbf{t}_i$ under depth bound H_i. This normalization accounts for the fact that multiple minimal representations might share the same support, This normalization accounts for the fact that multiple minimal representations might share the same support. Such pairs shared across multiple linear combinations are advantageous for our optimization, as target vectors containing these shared pairs typically exhibit smaller $\#avg_supp_i$ values. For a candidate pair p, we compute the total average support:~~ has a minimal representation of length 2, the corresponding pair is guaranteed to appear in that target's node subsequence. No estimation error can then occur, so such pairs should be prioritized.

$$Total_supp(p) = \sum \#avg_supp_i$$

~~We set a threshold and select all pairs p with $Total_supp(p)$ below this threshold. This approach prioritizes pairs associated with targetvectors that have smaller average support sizes.~~

~~In this section, we propose a new circuit representation method and formulate the circuit reconstruction problem based on this representation. We then introduce novel strategies for selecting subcircuits and present efficient reconstruction approaches.~~ ~~Currently, there are three primary methods for representing circuit implementations: the node/operation sequence [?, ?, ?], the tree/graph representation [?, ?] and the vector sequence [?]. However, these representation methods introduce specific representational redundancydue to their inherent structural constraints, including the order of operations/vectors and the generative relationships between nodes. That is , for circuits with identical functionality and identical output values at all intermediate nodes, multiple distinct representations can exist. For instance, in sequence-based representations, swapping the order of non-dependent operations may yield different sequences that , nonetheless, correspond to the exact same circuit implementation. Similarly, a circuit with a given set of intermediate output values can correspond to multiple different topological structures in tree or graph representations .~~

3.6 Subcircuit reconstruction

~~These redundant representations cause a critical issue during local optimization: the optimizer repeatedly attempts to optimize subcircuits that are functionally equivalent and share identical intermediate states, thereby significantly slowing down optimization efficiency. To address this problem, we propose a set representation.~~

~~**The set representation of circuits and reconstruction problem** The set representation of circuits relaxes the constraints on the generation order and methodology of vectors. Its core principle is that for a given set, as long as there existsa serialization order that allows the set to correspond to a valid CVS sequence, then that set can represent the circuits implemented by that CVS sequence. () Given an $B^{(l)} = \{\mathbf{b}_0, \cdots, \mathbf{b}_{l-1}\}$, the~~ The goal of subcircuit reconstruction is to select a subcircuit $\mathcal{C}$ and replace it with a new subcircuit $\mathcal{C}'$ that has fewer gates while preserving the same input and output

Function 4: The `ComputeScore` function

~~In addition, we propose a tie-breaking strategy called the *support overlap maximization* criterion. This criterion aims to ensure that in the next iteration, there are more pairs with higher frequency, thereby increasing the number of candidate pairs and expanding the search space without degrading solution quality. We observe that if there exists at least one pair with frequency greater than 1, then necessarily~~

$\cancel{\sum_{\mathbf{t}_i \in T} |\text{support}_{H_i}(B, \mathbf{t}_i)| > \left|\bigcup_{\mathbf{t}_i \in T} \text{support}_{H_i}(B, \mathbf{t}_i)\right|}$.

~~For a candidate pair $\{\mathbf{b}_i, \mathbf{b}_j\}$, we compute the new vector $\mathbf{b}_k = \mathbf{b}_i \oplus \mathbf{b}_j$ and predict the support sets for all target vectors after adding $\mathbf{b}_k$ to the sequence, i.e., $\text{support}_{H_i}(B \cup \{\mathbf{b}_k\}, \mathbf{t}_i)$. We then calculate the value:~~

$$\cancel{\sum_{\mathbf{t}_i \in T} |\text{support}_{H_i}(B \cup \{\mathbf{b}_k\}, \mathbf{t}_i)| - |\bigcup_{\mathbf{t}_i \in T} \text{support}_{H_i}(B \cup \{\mathbf{b}_k\}, \mathbf{t}_i)|}$$

~~and select the pair that maximizes this value. This approach prioritizes pairsthat maximize the overlap among support sets in the next iteration, promoting higher pair frequency and diversity in subsequent steps.~~ **Input** : $\mathcal{R}_H$: mapping from each target vector to its depth-constrained minimal-operation representations; $\mathcal{P}_{\texttt{valid}}$: set of valid pairs; $\ell \in \mathbb{N}$: length threshold.

~~Based on the new tie-breaking techniques and the target selection strategy, we present an improved algorithm for generating the depth-constrained extended in Appendix ??.~~

Output : *Score*: mapping from candidate pairs to their scores.

```
begin
    Score_{i,j} ← 0                                  // Score_{i,j} is the score of the pair {w_i, w_j}
    for (t, R_min) ∈ R_H do
        for each pair {w_i, w_j} ∈ P_valid do
            if exists L ∈ R_min s.t. {w_i, w_j} ⊂ L then
                if |L| = 2 then
                    Score_{i,j} ← +∞
                else if |L| ≤ ℓ and Score_{i,j} ≠ +∞ then
                    Score_{i,j} ← Score_{i,j} + 1
    for each pair {w_i, w_j} ∈ P_valid do
        Append ({w_i, w_j}, Score_{i,j}) to Score
    return Score
```

3.6 ~~The local optimization~~

nodes. A basic approach is to take a consecutive subsequence $\vec{S}_{\text{sub}} = [\mathbf{w}_i, \ldots, \mathbf{w}_{i+k-1}]$ from $\vec{S}$, treat $\vec{S}_{\text{prefix}} = X \cup \{\mathbf{w}_0, \ldots, \mathbf{w}_{i-1}\}$ as inputs and $\vec{S}_{\text{suffix}} = [\mathbf{w}_{i+k}, \ldots, \mathbf{w}_{s-1}]$ as targets, and run BP to find a shorter sequence. If successful, concatenating the prefix, the new sequence, and the suffix yields a shorter overall circuit. This method, however, restricts the subcircuit to nodes that appear consecutively in the original order.

To allow non consecutive selections, we propose a more flexible method for a given set C. We say that a set Z with k elements can be *arranged into a valid node sequence* if its vectors can be ordered as $\vec{Z} = [\mathbf{z}_0, \ldots, \mathbf{z}_{k-i}]$ such that each $\mathbf{z}_i$ satisfies $\mathbf{z}_i = \texttt{OP}(\mathbf{u}, \mathbf{v})$ with $\mathbf{u}, \mathbf{v}$ coming from X or from earlier vectors in $\vec{Z}$. For any Z, a maximal subset that can be so arranged can be computed. Let C be the vectors in subcircuit, we compute the set $\cancel{B^{(s)} = \{\mathbf{b}_l, \cdots, \mathbf{b}_{l+s-1}\}}$ ~~is an~~ ~~with respect to $B^{(l)}$, if there exists an $\mathcal{B}^{(s)}$ with respect to $B^{(l)}$ under an ordering such that $\text{set}(\mathcal{B}^{(s)}) = B^{(s)}$. Further more, let $T = \{\mathbf{t}_0, \cdots, \mathbf{t}_{m-1}\}$ be an . The $B^{(s)}$ is a~~ ~~for T if $T \subseteq B^{(s)}$~~ C_{prefix}, which is the maximal subset of $S \setminus C$ that can be arranged into a valid node sequence (its sequence is $\vec{S}_{\text{prefix}}$), and the set $C_{\text{suffix}} = S \setminus (C \cup C_{\text{prefix}})$. Replacing C does not affect the prefix. However, if $|C_{\text{suffix}}|$ is large, optimizing over all its vectors becomes inefficient. To improve efficiency, we select a smaller set $C_{\text{key}} \subseteq C_{\text{suffix}}$ as targets for reconstruction, leaving $C_{\text{tail}} = C_{\text{suffix}} \setminus C_{\text{key}}$ to be

arranged later. The condition is that C_{tail} must be arrangeable using $X \cup C_{\text{prefix}} \cup C_{\text{key}}$ as inputs. ~~Let $B^{(l)}$ be an . If B is a vector sum set with respect to $B^{(l)}$, and B' is a vector sum set with respect to $B^{(l)} \cup B$, then $B \cup B'$ is also a vector sum set with respect to $B^{(l)}$.~~

~~The is also referred to as a CVS set.Given a CVS set, we can generate a CVS sequence with the minimum possible depth using the algorithm **??** in Appendix **??**. The depth of this sequence is defined as the depth of the CVS set.~~

~~(The depth-constrained circuit reconstruction problem)Given an input generator set $B^{(l)} = \{\mathbf{b}_0, \cdots, \mathbf{b}_l\}$, an output target set T and an positive integer H. Let B be a CVS set for T with respect to $B^{(l)}$ and $\mathcal{D}(B) \leq H$. The problem is how to compute two set of vectors D and D' such that~~ The reconstruction then proceeds in four steps:

Step 1. ~~$D \subset B \setminus T$;~~ Compute the prefix set C_{prefix} and its sequence $\vec{S}_{\text{prefix}}$.

Setp 2. ~~$|D| > |D'|$;~~ Choose $C_{\text{key}} \subseteq C_{\text{suffix}}$; set $C_{\text{tail}} = C_{\text{suffix}} \setminus C_{\text{key}}$.

Step 3. ~~$B' = (B \setminus D) \cup D'$ is a for T with repect to $B^{(l)}$;~~ Arrange C_{tail} into a valid node sequence $\vec{S}_{\text{tail}}$ using $X \cup C_{\text{prefix}} \cup C_{\text{key}}$ as inputs.

Step 4. ~~$\mathcal{D}(B') \leq H$.~~ Take $X \cup C_{\text{prefix}}$ as inputs and C_{key} as targets, apply BP to obtain a new sequence $\vec{S}'$, and let C' be the set of vectors in $\vec{S}'$. The replacement for C is $C' \setminus C_{\text{key}}$.

~~**The new method for local optimization**~~ The reconstructed circuit sequence is $\vec{S}_{\text{prefix}} \mid \vec{S}' \mid \vec{S}_{\text{tail}}$.

~~This section defines the out-degree of a subset and, based on this concept, introduces a corresponding subset selection strategy. The methods presented herein can be directly generalized to circuit reconstruction problems without depth constraints, simply by setting the depth bound H to a sufficiently large integer.~~

~~(H-depth) Let $B^{(s)}$ be a CVS set for the output target~~ We use the following example to illustrate the process of subcircuit reconstruction. Consider the sequence from Tab. **??**. Choose $C = \{\mathbf{w}_{26}, \mathbf{w}_{27}\}$ for reconstruction. The prefix set $C_{\text{prefix}} = \{\mathbf{w}_0, \ldots, \mathbf{w}_{25}, \mathbf{w}_{30}, \ldots, \mathbf{w}_{35}\}$ and the remaining suffix is $C_{\text{suffix}} = \{\mathbf{w}_{28}, \mathbf{w}_{29}\}$. Since $\mathbf{w}_{29} = \mathbf{w}_{28} \oplus \mathbf{e}_4$, we set ~~$T$ with respect to the input generator set $B^{(l)}$. Given a set of vectors $D \subseteq B^{(s)} \setminus T$ and a positive integer H, the H-*depth* of D is the set $B^{(s)} \setminus (\text{useful}_H(D) \cup D)$, denoted $\text{affect}_H(D)$~~$C_{\text{key}} = \{\mathbf{w}_{28}\}$ and $C_{\text{tail}} = \{\mathbf{w}_{29}\}$. To keep $\mathbf{w}_{29}$ within the depth limit $H = 15$, we require $\mathcal{D}(\mathbf{w}_{28}) \leq 14$. Using C_{prefix} as inputs, a depth-constrained BP run finds $\mathbf{w}_{28} = \mathbf{w}_{30} \oplus \mathbf{w}_{33} \oplus \mathbf{w}_{25}$; only $\{\mathbf{w}_{30}, \mathbf{w}_{25}\}$ is a valid pair. Hence we set $C' = \{\mathbf{w}_{\text{new}} := \mathbf{w}_{30} \oplus \mathbf{w}_{25}\}$. Then $\mathbf{w}_{28} := \mathbf{w}_{\text{new}} \oplus \mathbf{w}_{33}$ (depth 14) and $\mathbf{w}_{29} := \mathbf{w}_{28} \oplus \mathbf{e}_4$ (depth 15) satisfy the constraint. The reconstructed sequence $[\mathbf{w}_0, \ldots, \mathbf{w}_{25}, \mathbf{w}_{30}, \ldots, \mathbf{w}_{35}, \mathbf{w}_{\text{new}}, \mathbf{w}_{28}, \mathbf{w}_{29}]$ has length 35, shorter than the original.

~~When the set $\text{affect}_H D$ is empty, it indicates that after removing set D, the corresponding useful set remains an H-depth CVS set for T with respect to $B^{(l)}$. Consequently, local optimization can be achieved by directly removing the vectors in D. In the context without depth constraints, the vectors eliminated by Rules 2-11 in [**?**] are precisely those for which the corresponding affected set is empty.~~

~~(H-depth minimal) Let $B^{(s)}$ be a CVS set for T with respect to $B^{(l)}$. Given a set of vectors $D \subseteq B^{(s)} \setminus T$ and a positive integer H, we call a set of vectors $\text{key}_H(D)$ the H-*depth* of D if it satisfies:~~

~~1. $\text{key}_H(D) \subseteq \text{affect}_H(D)$;~~

~~2. $B^{(u)}$ is a with respect to $\text{useful}_H(D) \cup \text{key}_H(D)$ where $B^{(u)} = B^{(s)} \setminus (D \cup \text{useful}_H(D) \cup \text{key}_H(D))$;~~

~~3. $\mathcal{D}(B^{(u)}) \leq H$.~~

~~Further more, the H-depth minimal of D is the H-depth with the minimal elements, denoted $\text{minkey}_H(D)$.~~

~~Usually, the minimal is generally not unique. The number of the H-depth minimal of D, denoted $\text{od}_H(D)$, is the **key set out-degree** of D. In graph representation, the out-degree of a node (vector) refers to the number of edges emanating from that node.~~

Function 5: The `SubcircuitReconstruct` function

~~Our approach proceeds as follows: Let D be the set of vectors to be reconstructed. We partition the CVS set into three disjoint subsets: D itself, the set of vectors that can be generated without D (called the *useful set*), and the set of vectors that cannot be generated without D (called the *affected set*, as they are directly affected by D). From the affected set, we then identify the minimal set of vectors that must be generated by D, termed the *minimal key set*. Thus, our criterion for subset selection is to find D such that the number of variables directly affected by its removal is minimized.~~

~~Below, we provide formal definitions of these sets and quantify the impact of D through the concept of *key set out-degree*. Finally, we present a strategy for selecting candidate subsets for reconstruction based on key set out-degree.~~ **Input** : $S \subseteq \mathbb{F}_2^n$: vector set in the circuit sequence; $X \subseteq \mathbb{F}_2^n$: initial input vector set; $R \subseteq \mathbb{F}_2^n$: input vectors of nonlinear gates; $G \subseteq \mathbb{F}_2^n$: output vectors of nonlinear gates; $H \in \mathbb{N}$: circuit depth limit.

~~(H-depth) Given an $B^{(l)}$, a set B and a positive integer H, if B' is the largest subset of B such that B' is a with respect to $B^{(l)}$ and $\mathcal{D}(B') \leq H$, then B' is the H-depth of B with respect to $B^{(l)}$. The *H-depth* of D, denoted $\text{useful}_H(D)$, is the H-depth of $B^{(s)} \setminus D$ with respect to $B^{(l)}$.~~ **Output :** $\vec{S}$: updated node sequence after subcircuit reconstruction.

~~Given an $B^{(l)}$, a set B and a positive integer H, the of B is unique and the definition of the is well defined.~~ **begin**

  $N \leftarrow$ the times limit for running; $t \leftarrow 0$; $\vec{S} \leftarrow$ empty sequence

  while $t < N$ **do**

    $C \leftarrow$ choose a subset of S `// C is the subcircuit to be reconstructed`

    Compute the sets C_{prefix}, C_{key} and C_{tail}

    $C' \leftarrow \texttt{BP}(X \cup C_{\text{prefix}}, R, G, C_{\text{key}}, H_{\text{key}})$ `// Take` $X \cup C_{\text{prefix}}$ `as inputs and` C_{key} `as targets,` H_{key} `is the depth limit of vectors in` C_{key}

    if $|C' \setminus C_{key}| < |C|$ **then**

      arrange the set $C_{\text{prefix}} \cup C' \cup C_{\text{key}} \cup C_{\text{tail}}$ into a valid node sequencev $\vec{S}$

      break

    $t \leftarrow t + 1$

  return $\vec{S}$

~~We can use the Algorithm 6 in [?] to compute the .~~

~~Notably, if a vector $\mathbf{b}$ has an out-degree of 1 in the graph representation, then the key set out-degree of the singleton set $\{\mathbf{b}\}$ is also 1.~~

~~Besides, the vectors that can be removed by the Rule 2-11 in [?], then the out-degree of D is 0.~~

~~Given the set of vectors $B^{(l)}$ and the output target set T, let $B^{(s)}$ be a CVS set for T with respect to $B^{(l)}$ and a positive integer H. For a set of vectors $D \subseteq B^{(s)} \setminus T$, if there exists a set of vectors D' such that :~~

~~1. D' is a vector with respect to $B^{(l)} \cup \text{useful}_H(D)$~~

~~2. $|D| > |D'|$,~~

~~3. $\text{minkey}_H(D)$ is a with respect to $B^{(l)} \cup \text{useful}_H(D) \cup D'$,~~

4 Results and applications

~~then $B^{(u)}$ is a CVS set for T with respect to $B^{(l)}$.~~ This section presents experimental results and analysis of the proposed General-BP framework and its improvement strategies. All experiments were conducted on an AMD Threadripper 3970X processor (32 cores) and 128 GB of RAM, running Ubuntu 20.04 LTS.

~~See Appendix ??.~~

~~Given a CVS set $B^{(s)}$ with depth H, to ensure that the optimized CVS set $B^{(u)}$ also has depth at most H, it suffices to guarantee that the depth of vectors in $\text{minkey}_H(D)$ does not increase. That is , for any CVS sequence $\mathcal{B}^{(s)}$ corresponding to $B^{(s)}$ with minimal depth, and for any $\mathbf{b} \in \text{minkey}_H(D)$, let $\mathcal{D}(\mathcal{B}^{(s)}, \mathbf{b})$ be the depth of $\mathbf{b}$ in $\mathcal{B}^{(s)}$. Then it is sufficient to ensure that the depth of $\mathbf{b}$ in the reconstructed fsequence $\mathcal{B}^{(u)}$~~

~~satisfies $\mathcal{D}(\mathcal{B}^{(u)}, \mathbf{b}) \leq \mathcal{D}(\mathcal{B}^{(s)}, \mathbf{b})$~~We first compare the running time of General-BP and Extended-BP [?], then introduce a new evaluation metric to compare different heuristic algorithms and evaluate the dynamic depth-limit allocation and weighted scoring strategies. Finally, we apply the complete framework to optimize `AES`, `SNOW3G`, and `Saturnin` S-box circuits. The circuits are provided in the supplementary materials. An anonymized GitHub repository containing the experimental code and reference circuit implementations used in this work is available at `https://github.com/Anonymous-user1234/Supplementary_Material`.

~~**The new strategy for selecting the subset** Given a CVS set B with depth H, the following procedure selects subsets and prefers local optimization:~~

4.1 Efficiency comparison

~~Step 1. For each vector $\mathbf{b} \in B$, compute its key set out-degree $\mathrm{od}_H(\{\mathbf{b}\})$. If $\mathrm{od}_H(\{\mathbf{b}\}) = 0$, remove $\mathbf{b}$ from B directly.~~ In this subsection, we compare General-BP with Extended-BP [?] . We evaluate three implementations: the public Python code of [?] (Extended-BP-Py, `https://github.com/lemontrr/Extended-BP-Framework`), our C++ reimplementation with the same algorithmic procedure (Extended-BP-C++), and our C++ implementation of General-BP. Tab. ?? reports single-thread running time under the same platform, instances, depth limit H, and parameter settings.

~~Step 2. Recalculate the H-depth key sets and key set out-degrees for all remaining vectors in B. Then, construct the set $D_1 = \{\mathbf{b} \mid \mathbf{b} \in B,\ \mathrm{od}_H(\{\mathbf{b}\}) = 1\}$, which comprises all vectors with an key set out-degree of 1.~~ Since BP-based algorithms use greedy selection with random tie-breaking, different experiments may produce different results. Therefore, we report the average over multiple independent experiments. General-BP is significantly faster, we perform $N = 10000$ experiments for each instance and depth limit. Extended-BP-Py and Extended-BP-C++ are much slower and show larger variation across experiments. For example, in the `AES` case with $H = 22$, the running time over 20 experiments ranges from 23,304.43 seconds (XOR count = 91) to 37,310.09 seconds (XOR count = 100). Therefore, for these slower variants, we report the average over $N = 20$ experiments. Tab. ?? summarizes the average running time under the above evaluation settings.

~~We first introduce a new evaluation metric for comparing different heuristic algorithms. Based on this metric,we evaluate various heuristic strategies and provide recommendations for strategy selection under different depth bounds. Finally,we apply our proposed methods to optimize the circuit implementations of `AES`,`SNOW3G`,and `Saturnin` S-boxes.~~ Tab. ?? shows that General-BP is about 10^4 times faster than Extended-BP-Py on all tested instances. Even when both methods are implemented in C++, General-BP remains significantly faster than Extended-BP-C++. Extended-BP-C++ follows the same algorithmic procedure as Extended-BP-Py and is about one order of magnitude faster, mainly due to programming language differences. General-BP achieves a further three orders of magnitude reduction in running time through algorithmic improvements.

4.2 A new metric for BP algorithm and its variants

~~Traditional evaluation methods for BP algorithms and their variants, such as comparing the minimal sequence length obtained after a fixed number of runs, suffer from inherent limitations due to randomness.~~ Since the search space of BP-based algorithms is ~~prohibitively large~~ huge and cannot be fully explored, ~~results from individual runs are heavily influenced by random factors, making such comparisons statistically unreliable~~different repeated experiments may produce different results due to randomness. Reporting only the best result may be misleading, and reporting only the average value reflects overall performance but cannot capture the ability to consistently achieve small `XOR` counts.

To address this issue, we propose a ~~well-defined evaluation metricbased on *cumulative distribution analysis*~~*cumulative distribution* metric. This metric analyzes the distribution of ~~solution~~ sequence quality across multiple independent ~~runs, plotting the cumulative percentage of runs that achieve a given performance level. For each algorithm variant, we execute~~ experiments. For N independent ~~runs and record the sequence length for each run. The cumulative distribution at sequence length~~ experiments, let q_i denote the `XOR` count in the i-th run, where $i \in 1, 2, \ldots, N$. For a given threshold q~~is calculated as:~~

$$CD(q) = (\text{Number of runs achieving sequence length} \leq q)/N \times 100\%.$$

, we compute $\mathrm{CD}(q) = \frac{|\{i|q_i \leq q\}|}{N} \times 100\%$.

~~A key insight is that if an algorithm can consistently find sequences of length~~ This metric measures the proportion of experiments achieving an `XOR` count no greater than q~~with higher probability, it is also more likely to discover sequences of length $q-1$ with additional runs. However, shorter sequences are observed less frequently and their occurrence is more susceptible to the influence of random seeds. By analyzing~~ . A high proportion at smaller q indicates more consistently achieve small `XOR` counts. Comparing the cumulative distribution ~~, we can reduce random variations and thus evaluate performance more reliably.~~ curves provides a more comprehensive view of performance and reduces the impact of randomness.

4.3 ~~Strategy comparison under depth constraints~~

~~In this subsection, we evaluate the effectiveness of different strategies using cumulative distribution analysis. We selected the `AES` S-box circuit from [?].~~

~~**Comparison of target strategies** We compared the performance between the strategy that considers all target vectors and the strategy that selectively filters target vectors. We use the maximum norm criterion for both strategies.~~

~~**All-target (AT) strategy** : At each step, all targetsare considered, used in [?].~~

~~**Selective-target (ST) strategy** : During frequency counting, target vectors are excluded if their minimal linear combination length exceeds $Bound_L = 8$ or their support set size exceeds $Bound_S = 40$.~~

~~Comparison of cumulative distributions between All-target(AT) and Selective-target(ST) strategies under different depth bounds~~

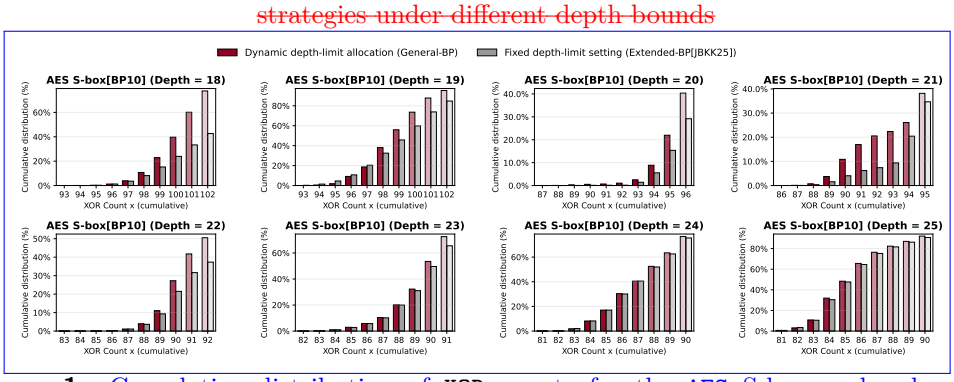

Figure 1: Cumulative distribution of `XOR` counts for the `AES` S-box under dynamic depth-limit allocation and the fixed depth-limit setting in [?].

~~Figure ?? presents our statistical results, showing the cumulative distribution of sequence lengths achieved by both strategies across 10000 independent runs.Experimental results indicate that the selective-target strategy performs better under smaller depth bounds,~~

~~while the all-target strategy is more effective when the depth bound is larger. This is reasonable because with a smaller depth bound, longer linear combinations with relatively small support sets have a lower probability of being further shortened. Consequently, such linear combinations are unlikely to remain the minimal representations consistently, and thus considering them may misguide the selection of pairs. In contrast, with a larger depth bound, these longer linear combinations have a higher probability of remaining as the minimal representations for the target vectors, thus considering them improves the accuracy of pair selection, leading to better performance of the AT strategy.~~

4.3 Impact of the dynamic depth-limit allocation strategy

~~Experimental results demonstrate that the selective-target strategy achieves better performance under smaller depthbounds, whereas the all-target strategy proves more effective when the depth bound is larger. This behavior can be explained by the fact that with tighter depth constraints, longer linear combinations possessing relatively small support sets have a reduced probability of being further shortened. Consequently, such linear combinations are less likely to persist as minimal representations throughout the optimization process, and their inclusion may misguide pair selection. Conversely, with more relaxed depth bounds, these longer linear combinations maintain a higher probability of remaining as minimal representations for target vectors. Therefore, considering them enhances pair selection accuracy, resulting in superior performance of the all-target strategy under these conditions~~When some targets do not reach their depth bounds, the algorithm reallocates the depth budget to increase the number of valid pairs for other targets, as described in Sec. ??. Fig. ?? compares the dynamic depth-limit allocation strategy with the fixed depth-limit setting of [?] on the AES implementation in [?]. For each depth limit, $N = 10000$ experiments are performed and the cumulative distribution of XOR counts is plotted, with XOR count q on the horizontal axis and $\mathrm{CD}(q)$ on the vertical axis.

~~Thus, for depth bounds close to the theoretical lower limit, we recommend using the ST strategy, while for higher depth bounds, we suggest employing the all-target strategy . This recommendation is supported by the experimental resultsshown in Figure ?? , where the selective strategy demonstrates clear advantages in stricter depth-constrained scenarios.~~

~~Comparison of tie-Breaking criteria~~ ~~We compare the effects of different tie-breaking criteria under various depth bounds. Since our strategies are designed for depth-constrained scenarios, we do not consider larger depth bounds in this experiment. We only consider depth bounds from 18~~ The horizontal axis is restricted to the interval from the minimum observed XOR count q_{min} to ~~23, using the Selective-Target strategy to filter target vectors, and compare different tie-breaking criteria. We consider the *Maximum Norm (MN)* criterion,~~ $q_{min} + 10$ at each depth, corresponding to the near-optimal region. Since the full range is not shown, the ~~*Average Support Minimization (ASM)* criterion, and~~ vertical axis may not reach 100%, and the maximum proportion can vary across depth limits. This reflects differences in the concentration of the result distributions.

Under relatively low depth limits (e.g., $H \leq 23$), this strategy significantly improves the results. In this case, the ~~*Support Overlap Maximization (SOM)* criterion~~depth limit is strict, and many candidate pairs become invalid. Reallocating the depth budget increases the number of valid pairs and enlarges the feasible search space. Under high depth limits (e.g., $H = 24, 25$), most candidate pairs already satisfy the depth requirement, so the effect of reallocation becomes small and the improvement is limited.

4.4 Impact of the weighted scoring strategy

As discussed in Sec. ??, minimal linear representations of target ~~vectors are easy to change as new vectors are added to the sequence, leading to prediction instability. The ASM~~

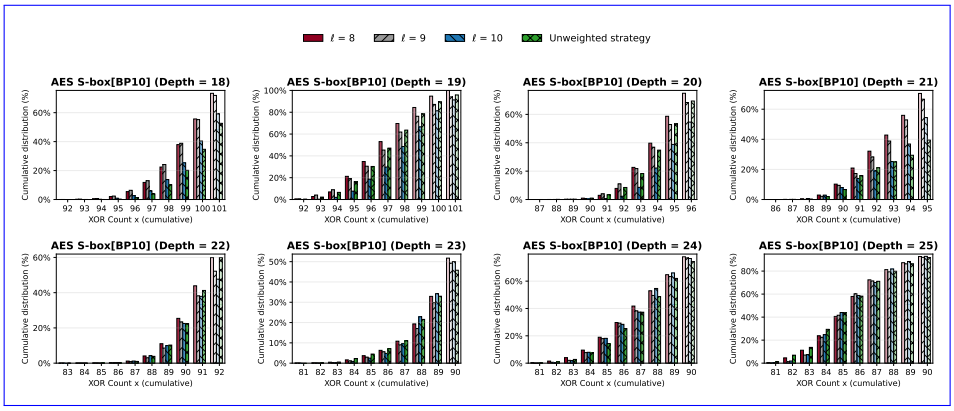

Figure 2: ~~Comparison~~ Cumulative distribution of ~~cumulative distributions among MN, ASM and SOM criteria~~ `XOR` counts for the `AES` S-box under different ~~depth bounds~~length thresholds ℓ and the unweighted strategy.

~~strategy prioritizes target vectors with smaller average support sizes. These vectors have relatively simpler and more stable linear representations, making pairs selected based on them more likely to remain valid. Under tight depth constraints, this stability reduces the risk of the algorithm being misled, thereby avoiding unnecessary increases in sequence length due to frequent changes in linear representations. In contrast, under depth bounds of 21 to 23, the SOM strategy demonstrates better performance. An explanation is that with larger depth bounds, the linear representations of target vectors tend to remain more stable. The SOM strategy selects pairs that maximize support overlap in the next iteration, meaning that a single pair can simultaneously reduce the distance of multiple target vectors. This increases the diversity of candidate pairs in subsequent iterations and expands the search space. Under looser depth constraints, such diversity helps the algorithm explore the global optimum more efficiently.Therefore, we recommend using the ASM criterion for tie-breaking when the depth bound is close to the theoretical lower limit, and employing the SOM strategy when the depth bound is relatively high.~~ may change during circuit construction, so candidate pairs counted in the expected usage may not appear in the final sequence. When all targets are included in scoring (i.e., the unweighted strategy in [**?**]), some pairs can be overestimated, leading to suboptimal decisions.

4.5 ~~Appications on `AES`, `SNOW3G`, and `Saturnin` S-Box Circuits~~

~~We evaluated the proposed algorithm in Subsection **??** on the `AES`, `SNOW3G`, and `Saturnin` S-boxes. All experiments were conducted on a system equipped with an AMD Threadripper 3970X processor (32 cores) and 128 GB of RAM, running Ubuntu 24.04 LTS. To ensure a fair comparison, we tested both the original Python implementation from [**?**] and our proposed method under the same experimental setup. For the `AES`, `Saturnin`, and `SNOW3G` S-boxes, our method achieved average runtimes of 0.22, 1.92, and 71.52 seconds, respectively. In comparison,~~ To reduce this discrepancy, we introduce a length threshold ℓ. A target contributes to the ~~original Python implementation of [**?**] required 24, 903 seconds, 135~~score only if its current representation length does not exceed ℓ; otherwise, its contribution is set to 0. This strategy limits the influence of long representations and reduces the gap between expected usage and actual usage.

Fig. **??** presents the cumulative distribution curves for different ℓ on the `AES` S-box implementation in [**?**]. For each depth limit, ~~231 seconds, and approximately one week for the same S-boxes in single runs.We also developed a C++ reimplementation of [**?**]'s algorithm, which yielded about a 10× speedup over the Python version. Even when~~

~~compared against this optimized C++ version, our algorithm still maintains a substantial efficiency advantage, corresponding to an overall improvement by a factor of approximately 10,000 relative to the original implementation~~$N = 10000$ independent experiments are performed, and the statistical method is the same as in Fig. ??. The figure compares $\ell = 8, 9, 10$ with the unweighted strategy. For this instance, the maximum representation length is 11, and about 76.5% of representations have length at most 8. Therefore, $\ell = 8$ corresponds to a strict filtering condition, while $\ell = 9$ and $\ell = 10$ are more relaxed.

From Fig. ??, the weighted strategy shows clear advantages under relatively low depth limits(e.g., $H \leq 21$), and the difference gradually decreases as the depth limit increases. Specifically, for strict depth limits (e.g., $H \leq 21$), $\ell = 8$ achieves the highest proportion in the near-optimal region. This is because feasible representations are limited, and long representations are more likely to be replaced by newly feasible alternatives during the iteration process. Since the weighted strategy emphasizes shorter representations, it more consistently achieves small `XOR` counts. This is consistent with the usage inconsistency discussed in Sec. ??.

As the depth limit increases (e.g., $H = 22, 23, 24, 25$), the advantage gets smaller. So feasible representations increases significantly. Even if representations change during the iterative process, there are usually several similar alternatives. Therefore, the gap between expected and actual usage does not increase significantly. In this case, Strict filtering may discard useful information, the advantage of the weighted strategy becomes small.

~~The detailed experimental results for~~

4.5 Applications on `AES`, `SNOW3G`, and `Saturnin` S-Box Circuits

In this subsection, we apply the proposed techniques to the `AES`, `SNOW3G`, and `Saturnin` ~~are summarized in Table ?? and ??, with all values representing the best solutions found within a two-hour execution time~~ S-box circuits and report the best results obtained within a five-hour time limit. The detailed comparison is given in Tab. ??.

The table compares the results in [?] with three configurations of our method: Sec. ??, which uses only the acceleration techniques with the original fixed depth-limit setting; Sec. ??-??, which additionally incorporates dynamic depth-limit allocation and weighted scoring; and Sec. ??, the complete General-BP framework including subgraph reconstruction. The threshold ℓ in the weighted strategy affects performance across instances, and for reproducibility we fix ℓ for each case, as listed in Tab. ??. ~~In Table ??, we compares the performance of our proposed strategies from Subsection ?? against the All-Target with Maximum Norm (AT+MN) criterion used in [?], applied to the baseline algorithm in Subsection ??. Our approach consistently outperforms conventional methods in most cases, validating its superior optimization capability for depth-constrained circuits.Table ??presents our best circuit implementations in comparison to those in [?]~~

It can be seen from Tab. ?? that progressively adding the proposed components leads to consistent improvements. We achieved improved implementations for every S-box instance originally reported in ~~that work~~[?], and extended the search to additional depth ~~constraints~~limits, discovering 17 previously unreported results among a total of 36. Notably, local optimization successfully shortened the circuit sequences in ~~24 of these~~ 23 of the 36 cases, ~~confirming~~ demonstrating its practical effectiveness~~—particularly~~. The improvement is especially significant for larger S-boxes~~, where~~. In particular, we report the first ~~depth-22~~ implementation of the `SNOW3G` S-box ~~, matching~~ with depth 22, which matches its theoretical minimum depth. ~~All resulting circuit data are provided in the supplementary materials, and the source code will be made publicly available upon publication.~~

5 Conclusion

~~Based on the work presented in this paper, we have significantly advanced the framework for optimizing the linear gates in S-box circuits. By formulating the problem as an Depth-Constrained Extended SLP problem and introducing novel concepts including H-depth minimal linear representations, H-feasible pairs, and H-depth support, we have developed an efficient algorithm that optimizes linear components in S-box implementations under strict depth constraints. Our approach incorporates new heuristic strategies for target vector selection and tie-breaking, along with effective local optimization techniques that eliminate redundant operations. Experimental results demonstrate that our method achieves superior optimization quality with a great improvement in computational efficiency compared to state-of-the-art approaches, while successfully generating optimized implementations for AES~~We propose a novel framework, General-BP, for solving depth-constrained circuit optimization problems. Through an in-depth analysis of BP-based depth-constrained optimization, ~~SNOW3G, and Saturnin S-boxes across various depth bounds.~~ we identify several key limitations and develop targeted improvements, which are integrated into the General-BP framework. These improvements include: storing depth-constrained information to avoid redundant computations, adopting a flexible dynamic depth-limit allocation strategy to increase the number of candidate pairs, introducing weighted scoring to better align evaluation scores with actual effects, and applying post-processing techniques to further refine the circuit. The effectiveness of these improvements is validated by experimental results, which demonstrate that General-BP not only achieves high efficiency but also finds superior solutions. We believe the limitations we have identified are of significant importance, and while this paper provides several effective solutions, they also open up possibilities for further research and alternative approaches.

A ~~Two deterministic algorithms for local optimization~~

~~**Case 1: Single Key Vector Optimization**~~

~~This case addresses the scenario where a set D has exactly one minimal key vector $\{\mathbf{b}\}$. The optimization exploits the linear representation of $\mathbf{b}$:~~

~~There are three prerequisites in this case:~~

~~1. D has key set out-degree 1 with minimal H-depth key set $\{\mathbf{b}\}$;~~

~~2. The minimal H_b-depth linear representation of $\mathbf{b}$ has size k where $H_b = \mathcal{D}(\mathcal{B}^{(s)}, \mathbf{b})$;~~

~~3. $|D| \geq k - 1$.~~

~~We can construct a replacement set D' of size $k-2$ that generates $\mathbf{b}$ from $\mathrm{useful}_H(D)$ with depth no larger than H_b. Since this sequence includes $\mathbf{b}$, we define $D' = \mathrm{set}(\mathcal{S}) \setminus \{\mathbf{b}\}$, and thus $|D'| = k-2$. Since $|D'| < |D|$, the length of the reconstructed CVS set is reduced, thereby achieving local optimization.~~

~~**Case 2: Shared Structure Optimization**~~

~~This case handles pairs of sets (D_1, D_2) that share computational substructure, enabling joint optimization:~~

~~The prerequisites in this case are as follows:~~

~~1. Both D_1 and D_2 have key set out-degree 1 and their H-depth minimal key sets are $\{\mathbf{b}_u\}$ and $\{\mathbf{b}_v\}$ respectively;~~

~~2. Their minimal linear representations share at least one feasible pair $\{\mathbf{b}_i, \mathbf{b}_j\}$. Specifically, the depths of vectors $\mathbf{b}_u$ and $\mathbf{b}_v$ are denoted as H_1 and H_2, respectively. Let L_1 and L_2 be the H_1-depth minimal linear representation of $\mathbf{b}_u$ and the H_2-depth minimal linear representation of $\mathbf{b}_v$ with respect to $\mathrm{useful}_H(D_1) \cap \mathrm{useful}_H(D_2)$, respectively. Then, the pair $\{\mathbf{b}_i, \mathbf{b}_j\}$ is both an H_1-feasible pair of L_1 and an H_2-feasible pair of L_2.~~

~~3. $|D_1| + |D_2| \geq k_1 + k_2 - 4$, where $k_1 = |L_1|$ and $k_2 = |L_2|$.~~

~~Then we can construct a replacement set D' of size $k_1 + k_2 - 5$ using the following concrete procedure. First, we add the vector $\mathbf{b} = \mathbf{b}_i \oplus \mathbf{b}_j$ to D', where $\{\mathbf{b}_i, \mathbf{b}_j\}$ is the common feasible pair. This allows us to transform the linear representations L_1 and L_2 into $L_1' = (L_1 \setminus \{\mathbf{b}_i, \mathbf{b}_j\}) \cup \{\mathbf{b}\}$ and $L_2' = (L_2 \setminus \{\mathbf{b}_i, \mathbf{b}_j\}) \cup \{\mathbf{b}\}$, with sizes $|L_1'| = k_1 - 1$ and $|L_2'| = k_2 - 1$ respectively. Subsequently, we construct sequences $\mathcal{S}_1$ and $\mathcal{S}_2$ with depths no greater than H_1 and H_2 respectively, where the lengths of these sequences are $k_1 - 3$ and $k_2 - 3$. The final replacement set is obtained as $D' = \{\mathbf{b}\} \cup \mathrm{set}(\mathcal{S}_1) \cup \mathrm{set}(\mathcal{S}_2)$, with cardinality $|D'| = k_1 + k_2 - 5$.~~

A Proofs

.1 ~~The proof of Proposition ??~~

~~We need to prove that $B^{(u)}$ is a CVS set for T with respect to $B^{(l)}$, i.e.,~~

~~(1). $T \subseteq B^{(u)}$,~~

~~(2). $B^{(u)}$ is a vector sum set with respect to $B^{(l)}$.~~

~~Proof of (1): $T \subseteq B^{(u)}$.~~

~~Since $D \subseteq B^{(s)} \setminus T$, it follows that $T \subseteq B^{(s)} \setminus D$. Therefore, $T \subseteq (B^{(s)} \setminus D) \cup D' = B^{(u)}$.~~

~~Proof of (2): $B^{(u)}$ is a sum set with respect to $B^{(l)}$ with depth at most H. Let $S_0 = B^{(l)}$, $S_1 = \mathrm{useful}_H(D)$, $S_2 = D'$, $S_3 = \mathrm{minkey}_H(D)$, $S_4 = \mathrm{affect}_H(D)$. According to Lemma ?? and definitions above, S_i is a vector sum set with respect to $\bigcup_{j=0}^{i-1} S_j$, so $\bigcup_{j=1}^{i-1} S_j$ is a vector sum set with respect to S_0. Since $B^{(u)} = S_1 \cup S_2 \cup S_3 \cup S_4$, we have $B^{(u)}$ is a vector sum set for T with respect to $B^{(l)}$.~~

.1 ~~The proof of Proposition ??~~

~~First, we prove $\text{useful}_H(D) = \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$ in two cases.~~

~~Case 1: $\text{useful}_H(D)$ is non-empty.~~

~~(1) Prove $\text{useful}_H(D) \subseteq \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$.~~

~~For any $\mathbf{b} \in \text{useful}_H(D)$, by the definition of the H-depth useful set, $\text{useful}_H(D)$ is the H-depth largest vector sum set of $B^{(s)} \setminus D$ with respect to $B^{(l)}$. Thus, there exists a vector sum set S with respect to $B^{(l)}$ (i.e., S is generated by a sum sequence of depth at most H) such that $\mathbf{b} \in S$ and $S \subseteq B^{(s)} \setminus D$.~~

~~Since $D = D_1 \cup D_2$, we have $S \subseteq B^{(s)} \setminus D_1$ and $S \subseteq B^{(s)} \setminus D_2$. By the definition of the useful set, this implies $S \subseteq \text{useful}_H(D_1)$ and $S \subseteq \text{useful}_H(D_2)$. Therefore, $\mathbf{b} \in \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$, and hence $\text{useful}_H(D) \subseteq \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$.~~

~~(2) Prove $\text{useful}_H(D_1) \cap \text{useful}_H(D_2) \subseteq \text{useful}_H(D)$.~~

~~For any $\mathbf{b} \in \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$, since $\mathbf{b} \in \text{useful}_H(D_1)$, there exists a vector sum set S_1 with respect to $B^{(l)}$ such that $\mathbf{b} \in S_1$ and $S_1 \subseteq \text{useful}_H(D_1)$.~~

~~Now, suppose every such vector sum set containing $\mathbf{b}$ intersects D_2, i.e., $S_1 \cap D_2 \neq \emptyset$. Then, $\mathbf{b}$ cannot belong to $\text{useful}_H(D_2)$ because $\text{useful}_H(D_2) \subseteq B^{(s)} \setminus D_2$. This contradicts $\mathbf{b} \in \text{useful}_H(D_2)$. Therefore, there exists a vector sum set S_1 satisfying $S_1 \subseteq \text{useful}_H(D_1)$ and $S_1 \cap D_2 = \emptyset$.~~

~~Since $S_1 \subseteq \text{useful}_H(D_1) \setminus D_2$ and $\text{useful}_H(D_1) \subseteq B^{(s)} \setminus D_1$, we have $S_1 \subseteq (B^{(s)} \setminus D_1) \setminus D_2 = B^{(s)} \setminus (D_1 \cup D_2)$. Thus, $S_1 \subseteq \text{useful}_H(D)$, and so $\mathbf{b} \in \text{useful}_H(D)$. This shows that $\text{useful}_H(D_1) \cap \text{useful}_H(D_2) \subseteq \text{useful}_H(D)$.~~

~~Case 2: $\text{useful}_H(D)$ is empty.~~

~~We need to prove that $\text{useful}_H(D_1) \cap \text{useful}_H(D_2)$ is also empty. Assume, for contradiction, that $\text{useful}_H(D_1) \cap \text{useful}_H(D_2)$ is non-empty. Then, there exists $\mathbf{b} \in \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$. Thus, there exists a vector sum set S with respect to $B^{(l)}$ such that $\mathbf{b} \in S$ and $S \subseteq \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$.~~

~~Since S is a vector sum set, it must contain some vectors generated by XOR operations on two vectors from $B^{(l)}$ (i.e., vectors of depth 1). Let $\mathbf{b}'_k$ be such a vector, so there exist $\mathbf{b}_i, \mathbf{b}_j \in B^{(l)}$ such that $\mathbf{b}'_k = \mathbf{b}_i \oplus \mathbf{b}_j$.~~

~~Since $\text{useful}_H(D)$ is empty, by the definition of the useful set, any vector generated from two vectors in $B^{(l)}$ (e.g., $\mathbf{b}'_k$) that belongs to $B^{(s)}$ must also belong to D (otherwise, $\mathbf{b}'_k$ would be in $\text{useful}_H(D)$). However, $\mathbf{b}'_k \in S \subseteq \text{useful}_H(D_1) \cap \text{useful}_H(D_2) \subseteq B^{(s)} \setminus D_1$ and $B^{(s)} \setminus D_2$, so $\mathbf{b}'_k \notin D_1$ and $\mathbf{b}'_k \notin D_2$. Therefore, $\mathbf{b}'_k \notin D = D_1 \cup D_2$, which contradicts $\mathbf{b}'_k \in D$. Hence, the assumption is false, and $\text{useful}_H(D_1) \cap \text{useful}_H(D_2)$ is empty.~~

~~Therefore, regardless of whether $\text{useful}_H(D)$ is empty or not, we have $\text{useful}_H(D) = \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$.~~

~~Next, we prove $\text{affect}_H(D) = (\text{affect}_H(D_1) \cup \text{affect}_H(D_2)) \setminus D$. By definition, we have: $\text{affect}_H(D) \cup D = B^{(s)} \setminus \text{useful}_H(D)$.~~

~~From the previously proven result $\text{useful}_H(D) = \text{useful}_H(D_1) \cap \text{useful}_H(D_2)$, we obtain: $B^{(s)} \setminus \text{useful}_H(D) = B^{(s)} \setminus (\text{useful}_H(D_1) \cap \text{useful}_H(D_2)) = (B^{(s)} \setminus \text{useful}_H(D_1)) \cup (B^{(s)} \setminus \text{useful}_H(D_2))$.~~

~~By definition, $B^{(s)} \setminus \text{useful}_H(D_1) = \text{affect}_H(D_1) \cup D_1$ and $B^{(s)} \setminus \text{useful}_H(D_2) = \text{affect}_H(D_2) \cup D_2$, so: $(B^{(s)} \setminus \text{useful}_H(D_1)) \cup (B^{(s)} \setminus \text{useful}_H(D_2)) = (\text{affect}_H(D_1) \cup D_1) \cup (\text{affect}_H(D_2) \cup D_2)$ $= \text{affect}_H(D_1) \cup \text{affect}_H(D_2) \cup D_1 \cup D_2$.~~

~~Since $D = D_1 \cup D_2$, we have $\text{affect}_H(D) \cup D = \text{affect}_H(D_1) \cup \text{affect}_H(D_2) \cup D$.~~

~~Since $\text{affect}_H(D)$ and D are disjoint by definition, so we have $\text{affect}_H(D) = (\text{affect}_H(D_1) \cup \text{affect}_H(D_2)) \setminus D$.~~

.1 ~~An MILP model for generating H-depth minimal linear combinations~~

~~An MILP model for generating H-depth minimal linear combinations~~

.1 ~~An improved algorithm for genenrating the depth-constrained extended~~

~~An improved algorithm for genenrating the depth-constrained extended~~

.1 ~~An algorithm to compute the CVS sequence of the given CVS set~~

~~An algorithm to compute the CVS sequence of the given CVS set.~~

Temporary page!

LaTeX was unable to guess the total number of pages correctly. As there was some unprocessed data that should have been added to the final page this extra page has been added to receive it.

If you rerun the document (without altering it) this surplus page will go away, because LaTeX now knows how many pages to expect for this document.